

KUBERNETES (K8S) TASK - 10

1.Setup EKS Cluster Using eksctl

Step 1: Prerequisites

Install/check:

`aws --version`

`kubectl version --client`

`eksctl version`

You should have:

- AWS CLI
- kubectl
- eksctl
- AWS account
- IAM permissions to create EKS, EC2, VPC, IAM, CloudFormation resources

Configure AWS:

`aws configure`

Enter:

AWS Access Key ID:

AWS Secret Access Key:

Default region name: ap-south-1

Default output format: json

Verify:

`aws sts get-caller-identity`

You should see your AWS account/user/role information.

```
MINGW64:/c/Users/braj/Downloads
brajk@RajKumari MINGW64 ~/Downloads
$ kubectl version --client
Client Version: v1.34.1
Kustomize Version: v5.7.1

brajk@RajKumari MINGW64 ~/Downloads
$ aws configure
AWS Access Key ID [*****35GO]: AKIA36JS5AW77HBT35GO
AWS Secret Access Key [*****shtc]: vQ05ocVcAoivAE7hoqqiw4nArsqLNKQK1T
swhstc
Default region name [us-east-1]:
Default output format [json]:

brajk@RajKumari MINGW64 ~/Downloads
$ aws sts get-caller-identity
{
  "UserId": "820982384063",
  "Account": "820982384063",
  "Arn": "arn:aws:iam::820982384063:root"
}

brajk@RajKumari MINGW64 ~/Downloads
```

Step 2: Create EKS cluster using eksctl

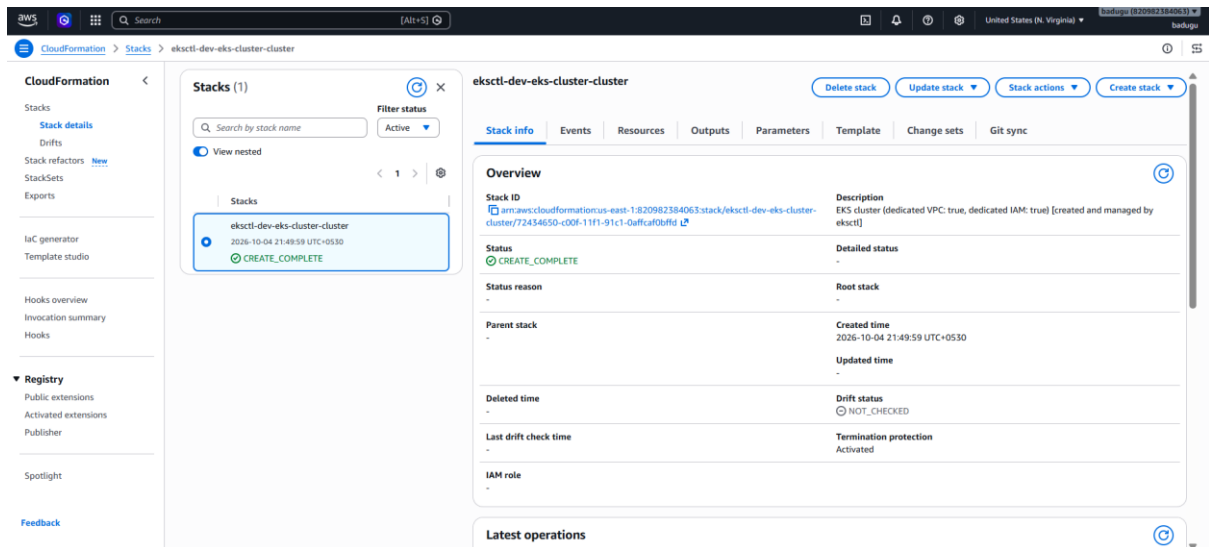
Create a configuration file:

nano eks-cluster.yaml

```
MINGW64:/c/Users/braj/eks-labs
GNU nano 6.3 eks-cluster.yaml
apiVersion: eksctl.io/v1alpha5
kind: ClusterConfig

metadata:
  name: dev-eks-cluster
  region: us-east-1
  version: "1.34"

managedNodeGroups:
- name: dev-ng
  instanceType: t3.medium
  desiredCapacity: 2
  minSize: 1
  maxSize: 4
```



Installation Steps for eksctl

MINGW64:/c/Users/brajk/Downloads

```
brajk@RajKumari MINGW64 ~/Downloads
$ uname -m
x86_64

brajk@RajKumari MINGW64 ~/Downloads
$ curl -LO "https://github.com/eksctl-io/eksctl/releases/latest/download/eksctl_
Windows_amd64.zip"
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total   Spent    Left   Speed
  0     0    0     0    0     0      0      0  --:--:--  0:00:02  --:--:--    0
  0     0    0     0    0     0      0      0  --:--:--  0:00:03  --:--:--    0
100 36.9M 100 36.9M    0     0 163k      0  0:03:51  0:03:51  --:--:-- 242k

brajk@RajKumari MINGW64 ~/Downloads
$ ls -lh eksctl_windows_amd64.zip
-rw-r--r-- 1 brajk 197609 37M Oct  4 21:20 eksctl_windows_amd64.zip

brajk@RajKumari MINGW64 ~/Downloads
$ mkdir -p ~/bin

brajk@RajKumari MINGW64 ~/Downloads
$ unzip -o eksctl_windows_amd64.zip -d ~/bin
Archive:  eksctl_windows_amd64.zip
  inflating: /c/Users/brajk/bin/eksctl.exe

brajk@RajKumari MINGW64 ~/Downloads
$ ls -l ~/bin/eksctl.exe
-rwxr-xr-x 1 brajk 197609 154797568 Oct  3 02:38 /c/Users/brajk/bin/eksctl.exe*

brajk@RajKumari MINGW64 ~/Downloads
$ echo 'export PATH="$HOME/bin:$PATH"' >> ~/.bashrc

brajk@RajKumari MINGW64 ~/Downloads
$ source ~/.bashrc

brajk@RajKumari MINGW64 ~/Downloads
$ eksctl version
0.231.0

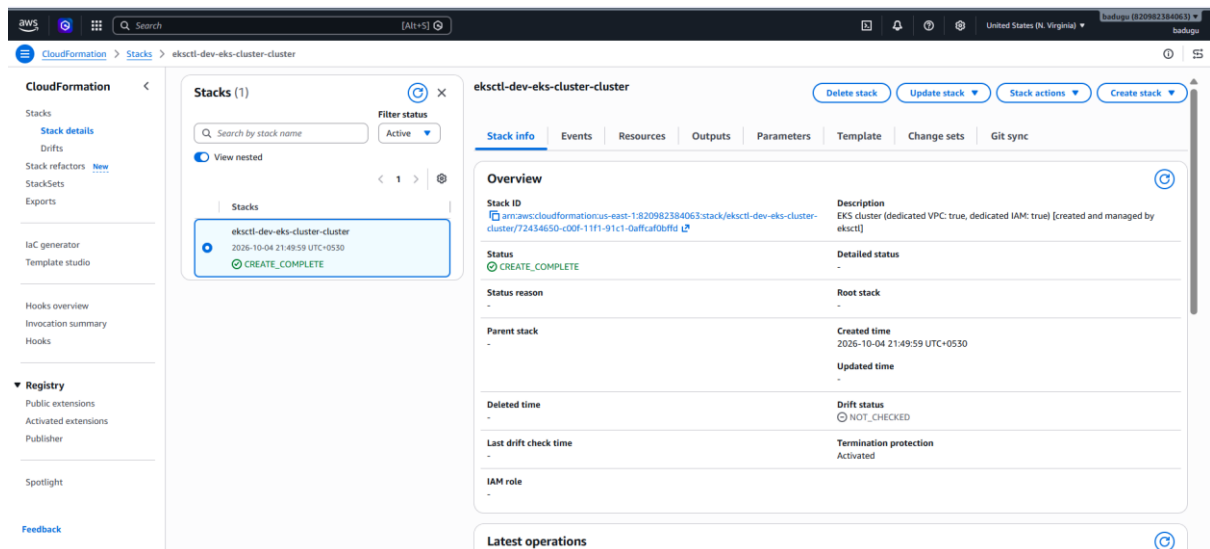
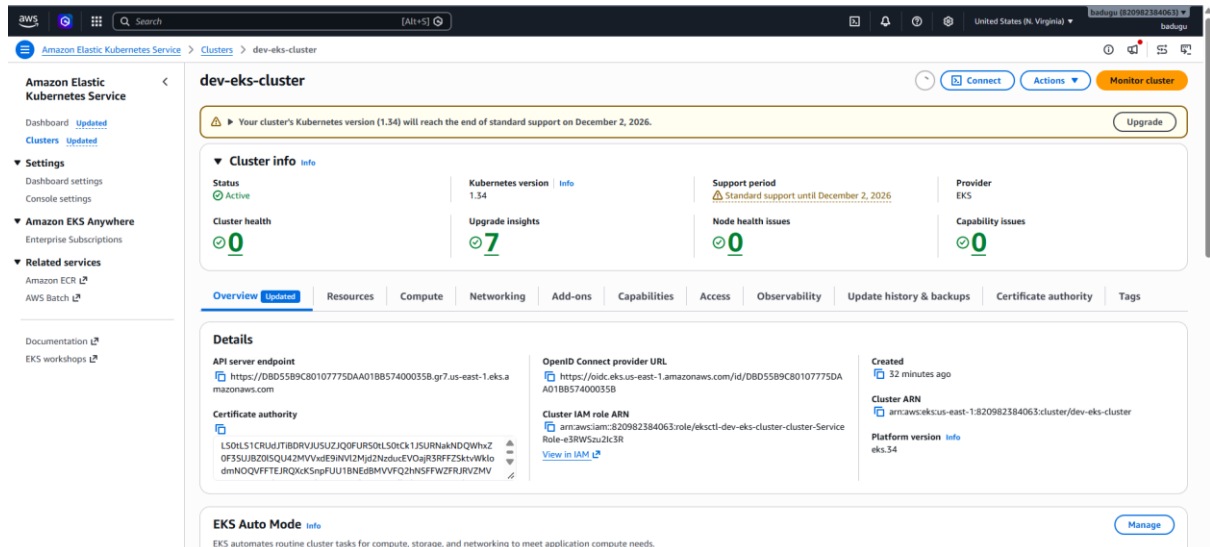
brajk@RajKumari MINGW64 ~/Downloads
$ which eksctl
/c/Users/brajk/bin/eksctl

brajk@RajKumari MINGW64 ~/Downloads
$ |
```

Create cluster:

```
eksctl create cluster -f eks-cluster.yaml
```

This can take several minutes.



Step 3: Verify cluster

```
eksctl get cluster
```

Then:

```
kubectl get nodes
```

Expected:

NAME	STATUS	ROLES
ip-10-xxx-xx-xx...	Ready	<none>

ip-10-xxx-xx-xx... Ready <none>

Step 4: Check namespaces

```
kubectl get ns
```

Step 5: Check system Pods

```
kubectl get pods -A
```

You should see components such as:

kube-system

including networking, DNS, kube-proxy, etc.

Step 6: Check EKS cluster

```
aws eks describe-cluster \
```

```
--name dev-eks-cluster \
```

```
--region ap-south-1
```

Step 7: Delete cluster after lab

This is important because EKS resources can incur AWS charges.

```
eksctl delete cluster \
```

```
--name dev-eks-cluster \
```

```
--region ap-south-1
```

Wait until:

```
eksctl get cluster
```

shows no cluster.

2.setup eks cluster using console

Prerequisites

Before creating the cluster, make sure you have:

On your Windows machine

You already have kubectl and eksctl from your previous EKS work, but for this lab we **will not use eksctl to create the cluster**.

Check:

kubectl version --client

You should see something similar to:

Client Version: v1.37.x

AWS CLI is also useful:

aws --version

Configure credentials if needed:

aws configure

Enter:

AWS Access Key ID:

AWS Secret Access Key:

Default region name: us-east-1

Default output format: json

If you're using an IAM role/SSO instead of access keys, use that authentication method instead.

2. Open AWS Console

Go to:

[AWS Management Console](#)

Sign in.

At the top-right, select:

Region → US East (N. Virginia) us-east-1

You can use another region, but I'll use **us-east-1** throughout this lab.

```
MINGW64:/c/Users/brajk/Downloads
brajk@RajKumari MINGW64 ~/Downloads
$ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
ip-10-0-16-110.ec2.internal        Ready    <none>   15m   v1.36.4-eks-3b4a6ca
ip-10-0-4-11.ec2.internal          Ready    <none>   15m   v1.36.4-eks-3b4a6ca

brajk@RajKumari MINGW64 ~/Downloads
$ |
```

```
MINGW64:/c/Users/brajk/Downloads
brajk@RajKumari MINGW64 ~/Downloads
$ aws eks update-kubeconfig --region us-east-1 --name eks-cluster
Added new context arn:aws:eks:us-east-1:820982384063:cluster/eks-cluster to C:\Users\brajk\.kube\config

brajk@RajKumari MINGW64 ~/Downloads
$ kubectl config get-contexts
CURRENT  NAME                                NAMESPACE          AUTHINFO            CLUSTER
         arn:aws:eks:us-east-1:820982384063:cluster/dev-eks-cluster  arn:aws:eks:us-east-1:820982384063:cluster/dev-eks-cluster
         arn:aws:eks:us-east-1:820982384063:cluster/eks-cluster  arn:aws:eks:us-east-1:820982384063:cluster/eks-cluster
         minikube
         default              minikube            minikube

brajk@RajKumari MINGW64 ~/Downloads
$ kubectl get nodes
No resources found

brajk@RajKumari MINGW64 ~/Downloads
$ s|
```

3.Setup HPA

Verify Your EKS Cluster

First make sure your EKS cluster is connected.

Open **Git Bash / PowerShell**.

Run:

```
kubectl get nodes
```

Step 2 — Verify Kubernetes Pods

Run:

```
kubectl get pods -A
```

You should see Kubernetes system components.

For example:

```
kube-system
```

```
aws-node
```

```
coredns
```

```
kube-proxy
```

Step 3 — Check Metrics Server

This is **very important**.

HPA needs CPU/memory metrics.

Run:

```
kubectl get deployment metrics-server -n kube-system
```

MINGW64:c:/Users/braj/k/Downloads

brajk@RajKumari MINGW64 ~/Downloads

\$ kubectl get nodes

NAME	STATUS	ROLES	AGE	VERSION
ip-10-0-16-110.ec2.internal	Ready	<none>	15m	v1.36.4-eks-3b4a6ca
ip-10-0-4-11.ec2.internal	Ready	<none>	15m	v1.36.4-eks-3b4a6ca

brajk@RajKumari MINGW64 ~/Downloads

\$ kubectl get pods -A

NAMESPACE	NAME	READY	STATUS	RESTARTS	AGE
kube-system	aws-node-8dgmw	2/2	Running	0	49m
kube-system	aws-node-wlwf	2/2	Running	0	49m
kube-system	coredns-79f8dffc57-7j8vp	0/1	Pending	0	3h4
kube-system	coredns-79f8dffc57-lz48b	1/1	Running	0	3h4
kube-system	eks-pod-identity-agent-pjps9	1/1	Running	0	49m
kube-system	eks-pod-identity-agent-vzj7f	1/1	Running	0	49m
kube-system	kube-proxy-8cgqh	1/1	Running	0	49m
kube-system	kube-proxy-vgqc4	1/1	Running	0	49m
kube-system	metrics-server-6877797457-5fgdj	1/1	Running	0	3h4
kube-system	metrics-server-6877797457-vmktw	0/1	Pending	0	3h4

brajk@RajKumari MINGW64 ~/Downloads

\$ kubectl get deployment metrics-server -n kube-system

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
metrics-server	1/2	2	1	3h45m

brajk@RajKumari MINGW64 ~/Downloads

\$ kubectl top nodes

NAME	CPU(cores)	CPU(%)	MEMORY(bytes)	MEMORY(%)
ip-10-0-16-110.ec2.internal	25m	1%	366Mi	71%
ip-10-0-4-11.ec2.internal	24m	1%	383Mi	74%

brajk@RajKumari MINGW64 ~/Downloads

\$ kubectl top pods -A

NAMESPACE	NAME	CPU(cores)	MEMORY(bytes)
kube-system	aws-node-8dgmw	3m	43Mi
kube-system	aws-node-wlwf	3m	43Mi
kube-system	coredns-79f8dffc57-lz48b	1m	11Mi
kube-system	eks-pod-identity-agent-pjps9	1m	6Mi
kube-system	eks-pod-identity-agent-vzj7f	1m	7Mi
kube-system	kube-proxy-8cgqh	2m	13Mi
kube-system	kube-proxy-vgqc4	1m	19Mi
kube-system	metrics-server-6877797457-5fgdj	3m	18Mi

brajk@RajKumari MINGW64 ~/Downloads

\$ |

Step 4 — If kubectl top Doesn't Work

If you get:

error: Metrics API not available

don't create the HPA yet.

Check:

kubectl get pods -n kube-system | grep metrics

Then:

kubectl describe deployment metrics-server -n kube-system

Check:

kubectl get apiservice | grep metrics

You want something similar to:

v1beta1.metrics.k8s.io

with:

AVAILABLE = True

EKS recommendation

For EKS, use the **Metrics Server add-on/configuration appropriate to your current EKS version** rather than blindly applying an old internet YAML.

In the AWS Console, you can check:

EKS

→ Clusters

→ Your cluster

→ Add-ons

Look for:

Metrics Server

If it isn't installed, we can install it before continuing.

Step 5 — Create Namespace

Let's create a separate namespace for our HPA lab.

```
kubectl create namespace hpa-demo
```

Check:

```
kubectl get namespaces
```

You should see:

hpa-demo

```
MINGW64:/c/Users/brajk/Downloads  
brajk@RajKumari MINGW64 ~/Downloads  
$ kubectl create namespace hpa-demo  
namespace/hpa-demo created  
brajk@RajKumari MINGW64 ~/Downloads  
$ kubectl get namespaces  
NAME                STATUS    AGE  
default             Active    3h59m  
hpa-demo            Active    8s  
kube-node-lease     Active    3h59m  
kube-public         Active    3h59m  
kube-system         Active    3h59m  
brajk@RajKumari MINGW64 ~/Downloads  
$
```

Step 6 — Create Deployment

Create a file:

hpa-deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hpa-demo
  namespace: hpa-demo
spec:
  replicas: 2
  selector:
    matchLabels:
      app: hpa-demo
  template:
    metadata:
      labels:
        app: hpa-demo
    spec:
      containers:
        - name: nginx
          image: nginx:latest
          resources:
            requests:
              cpu: 100m
              memory: 128Mi
            limits:
              cpu: 500m
              memory: 256Mi
          ports:
            - containerPort: 80
```

Step 7 – Understand the Important Part

This section is critical:

resources:

requests:

cpu: 100m

HPA calculates CPU utilization relative to the **CPU request**.

For example:

CPU request = 100m

CPU usage = 50m

CPU utilization = 50%

Therefore:

CPU request

↓

100m

↓

50m used

↓

50% utilization

That's why we should define CPU requests when using CPU-based HPA.

Step 8 — Create Deployment

Run:

```
kubectl apply -f hpa-deployment.yaml
```

Expected:

```
deployment.apps/hpa-demo created
```

Step 9 — Verify Deployment

Run:

```
kubectl get deployment -n hpa-demo
```

Expected:

NAME	READY	UP-TO-DATE	AVAILABLE
hpa-demo	2/2	2	2

Step 10 — Check Pods

```
kubectl get pods -n hpa-demo -o wide
```

Expected:

NAME	READY	STATUS
------	-------	--------

hpa-demo-xxxxxxxxxx-xxxxx 1/1 Running

hpa-demo-xxxxxxxxxx-yyyyy 1/1 Running

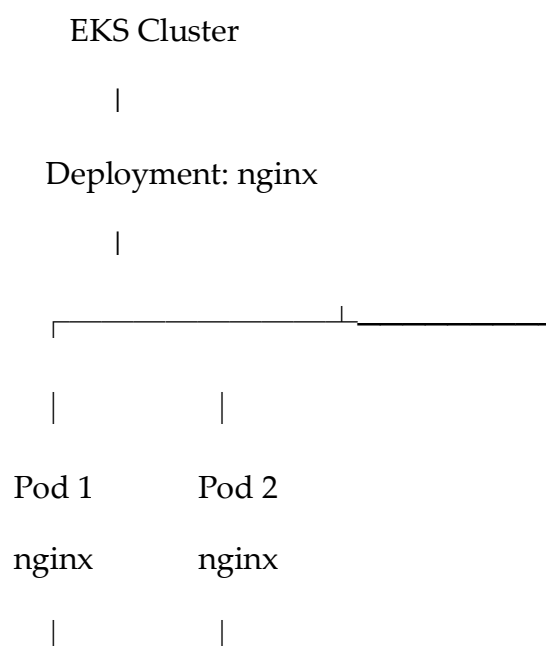
You have:

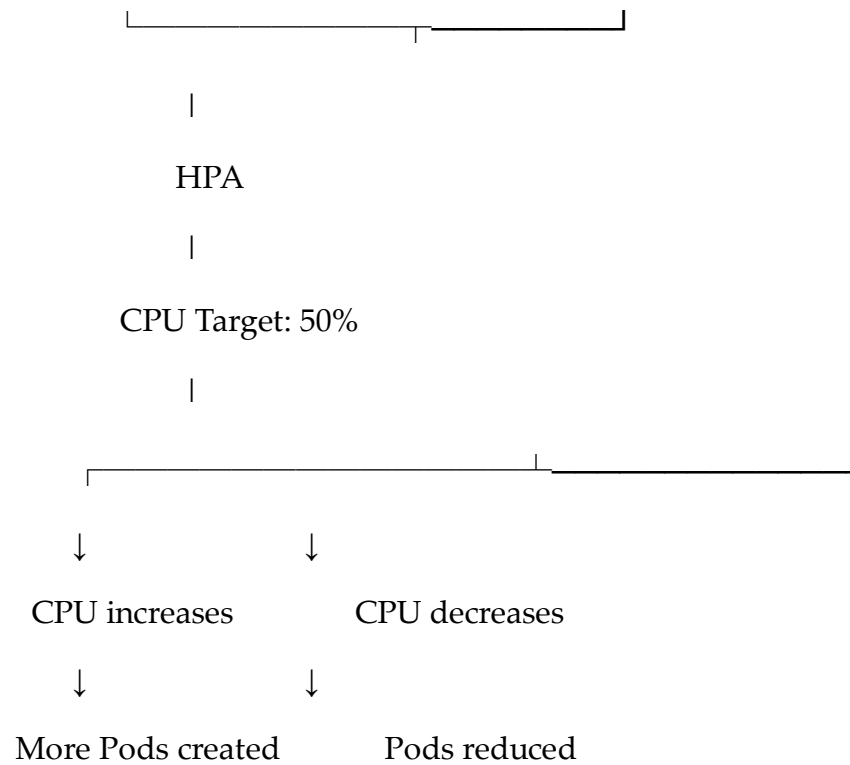
2 Pods

```
MINGW64:/c:/Users/brajk/Downloads
brajk@RajKumari MINGW64 ~/Downloads
$ kubectl get nodes -o wide
NAME                                STATUS    ROLES    AGE     VERSION    INTERNAL-IP    EXTERNAL-IP    OS-IMAGE             CONTAINER-RUNTIME
ip-10-0-16-110.ec2.internal         Ready    <none>    87m     v1.36.4-eks-3b4a6ca    10.0.16.110    13.217.160.245    Amazon Linux 2023.x86_64 (amd64)    containerd://2.2.7+unknown
ip-10-0-31-185.ec2.internal         Ready    <none>    4m38s   v1.36.4-eks-3b4a6ca    10.0.31.185    13.222.34.52     Amazon Linux 2023.x86_64 (amd64)    containerd://2.2.7+unknown
ip-10-0-4-11.ec2.internal           Ready    <none>    87m     v1.36.4-eks-3b4a6ca    10.0.4.11     100.52.230.101    Amazon Linux 2023.x86_64 (amd64)    containerd://2.2.7+unknown
brajk@RajKumari MINGW64 ~/Downloads
$ kubectl get pods -n hpa-demo -o wide
NAME                                READY    STATUS    RESTARTS   AGE    IP            NODE
hpa-demo-69bd5b979-jhg6c            0/1     Pending   0           24m    <none>        <none>
hpa-demo-69bd5b979-s7z84            0/1     Pending   0           24m    <none>        <none>
brajk@RajKumari MINGW64 ~/Downloads
$ |
```

Setup HPA on EKS — Pin-to-Pin Lab

What we are going to build





Example:

Minimum replicas = 2

Maximum replicas = 10

CPU target = 50%

Step 1 — Verify Your EKS Cluster

First make sure your EKS cluster is connected.

Open **Git Bash** / **PowerShell**.

Run:

```
kubectl get nodes
```

Expected:

NAME	STATUS	ROLES	AGE
ip-10-0-xxx-xxx.ec2.internal	Ready	<none>	...
ip-10-0-xxx-xxx.ec2.internal	Ready	<none>	...

All nodes should be:

Ready

If they are not Ready, fix that first.

Step 2 — Verify Kubernetes Pods

Run:

```
kubectl get pods -A
```

You should see Kubernetes system components.

For example:

```
kube-system
```

```
aws-node
```

```
coredns
```

```
kube-proxy
```

Step 3 — Check Metrics Server

This is **very important**.

HPA needs CPU/memory metrics.

Run:

```
kubectl get deployment metrics-server -n kube-system
```

If Metrics Server exists, you may see:

NAME	READY	UP-TO-DATE	AVAILABLE
metrics-server	1/1	1	1

Now run:

```
kubectl top nodes
```

Expected:

NAME	CPU(cores)	CPU%	MEMORY(bytes)	MEMORY%
ip-10-0-xxx-xxx.ec2.internal	150m	7%	1200Mi	30%

ip-10-0-xxx-xxx.ec2.internal 100m 5% 1100Mi 27%

And:

kubectl top pods -A

You should see CPU and memory usage.

Step 4 — If kubectl top Doesn't Work

If you get:

error: Metrics API not available

don't create the HPA yet.

Check:

kubectl get pods -n kube-system | grep metrics

Then:

kubectl describe deployment metrics-server -n kube-system

Check:

kubectl get apiservice | grep metrics

You want something similar to:

v1beta1.metrics.k8s.io

with:

AVAILABLE = True

EKS recommendation

For EKS, use the **Metrics Server add-on/configuration appropriate to your current EKS version** rather than blindly applying an old internet YAML.

In the AWS Console, you can check:

EKS

→ Clusters

→ Your cluster

→ Add-ons

Look for:

Metrics Server

If it isn't installed, we can install it before continuing.

Step 5 — Create Namespace

Let's create a separate namespace for our HPA lab.

```
kubectl create namespace hpa-demo
```

Check:

```
kubectl get namespaces
```

You should see:

```
hpa-demo
```

Step 6 — Create Deployment

Create a file:

```
hpa-deployment.yaml
```

Put this inside:

```
apiVersion: apps/v1
```

```
kind: Deployment
```

```
metadata:
```

```
  name: hpa-demo
```

```
  namespace: hpa-demo
```

```
spec:
```

```
  replicas: 2
```

```
  selector:
```

```
    matchLabels:
```

```
  app: hpa-demo
template:
  metadata:
    labels:
      app: hpa-demo
  spec:
    containers:
      - name: nginx
        image: nginx:latest
        resources:
          requests:
            cpu: 100m
            memory: 128Mi
          limits:
            cpu: 500m
            memory: 256Mi
        ports:
          - containerPort: 80
```

Step 7 — Understand the Important Part

This section is critical:

```
resources:
  requests:
    cpu: 100m
```

HPA calculates CPU utilization relative to the **CPU request**.

For example:

CPU request = 100m

CPU usage = 50m

CPU utilization = 50%

Therefore:

CPU request

↓

100m

↓

50m used

↓

50% utilization

That's why we should define CPU requests when using CPU-based HPA.

Step 8 — Create Deployment

Run:

```
kubectl apply -f hpa-deployment.yaml
```

Expected:

```
deployment.apps/hpa-demo created
```

Step 9 — Verify Deployment

Run:

```
kubectl get deployment -n hpa-demo
```

Expected:

NAME	READY	UP-TO-DATE	AVAILABLE
hpa-demo	2/2	2	2

Step 10 — Check Pods

kubectl get pods -n hpa-demo -o wide

Expected:

NAME	READY	STATUS
hpa-demo-xxxxxxxxxx-xxxxx	1/1	Running
hpa-demo-xxxxxxxxxx-yyyyy	1/1	Running

You have:

2 Pods

Step 11 — Create HPA

Now create the Horizontal Pod Autoscaler.

Run:

```
kubectl autoscale deployment hpa-demo \
  --namespace=hpa-demo \
  --cpu-percent=50 \
  --min=2 \
  --max=10
```

Expected:

horizontalpodautoscaler.autoscaling/hpa-demo autoscaled

Step 12 — Check HPA

Run:

kubectl get hpa -n hpa-demo

You should see something like:

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
------	-----------	---------	---------	---------	----------

hpa-demo Deployment/hpa-demo 5%/50% 2 10 2

Meaning:

Current CPU = 5%

Target CPU = 50%

Minimum pods = 2

Maximum pods = 10

Current pods = 2

Step 13 — Understand HPA

Our configuration is:

CPU target = 50%

Min = 2

Max = 10

Suppose:

CPU utilization = 20%

HPA says:

CPU is below target

↓

Keep 2 pods

If CPU increases:

CPU = 80%

↓

Above 50%

↓

HPA increases replicas

For example:

2 Pods

↓

4 Pods

↓

6 Pods

until CPU comes closer to the target or the maximum of 10 is reached.

Step 14 — Generate CPU Load

Our nginx pods aren't doing enough CPU work naturally.

So we'll create a temporary load-generator pod.

Create:

load-generator.yaml

Use:

apiVersion: v1

kind: Pod

metadata:

 name: load-generator

 namespace: hpa-demo

spec:

 containers:

 - name: load-generator

 image: busybox:1.36

 command:

 - /bin/sh

```
- -c
- |
while true; do
    wget -q -O- http://hpa-demo-service;
done
```

But first we need a Service.

Step 15 — Create Service

Create:

hpa-service.yaml

apiVersion: v1

kind: Service

metadata:

name: hpa-demo-service

namespace: hpa-demo

spec:

selector:

app: hpa-demo

ports:

- port: 80

targetPort: 80

Apply it:

kubectl apply -f hpa-service.yaml

Check:

kubectl get svc -n hpa-demo

Expected:

NAME	TYPE	CLUSTER-IP
hpa-demo-service	ClusterIP	10.x.x.x

Step 16 — Create Load Generator

Now apply:

```
kubectl apply -f load-generator.yaml
```

Check:

```
kubectl get pod -n hpa-demo
```

You should see:

```
hpa-demo-xxxxx
```

```
hpa-demo-yyyyy
```

```
load-generator
```

Step 17 — Watch HPA

Run:

```
kubectl get hpa -n hpa-demo -w
```

You may initially see:

NAME	REFERENCE	TARGETS	MINPODS	MAXPODS	REPLICAS
hpa-demo	Deployment/hpa-demo	5%/50%	2	10	2

As CPU increases, you may eventually see:

hpa-demo	Deployment/hpa-demo	80%/50%	2	10	4
----------	---------------------	---------	---	----	---

Then:

hpa-demo	Deployment/hpa-demo	65%/50%	2	10	4
----------	---------------------	---------	---	----	---

The exact numbers and timing will vary.

Step 18 — Watch Pods

Open another terminal.

Run:

```
kubectl get pods -n hpa-demo -w
```

You may see:

2 Pods

↓

4 Pods

↓

6 Pods

depending on the load.

Step 19 — Check Deployment

Run:

```
kubectl get deployment -n hpa-demo
```

For example:

NAME	READY	UP-TO-DATE	AVAILABLE
------	-------	------------	-----------

hpa-demo	4/4	4	4
----------	-----	---	---

The HPA has changed the Deployment's desired replica count.

Step 20 — See HPA Details

Run:

```
kubectl describe hpa hpa-demo -n hpa-demo
```

Look for:

Min replicas: 2

Max replicas: 10

CPU utilization: ...

Target CPU utilization: 50

Also check the events at the bottom.

For example:

SuccessfulRescale

This tells you when HPA scaled the workload.

Step 21 — Stop the Load

Delete the load generator:

```
kubectl delete pod load-generator -n hpa-demo
```

Now CPU should decrease.

Check:

```
kubectl top pods -n hpa-demo
```

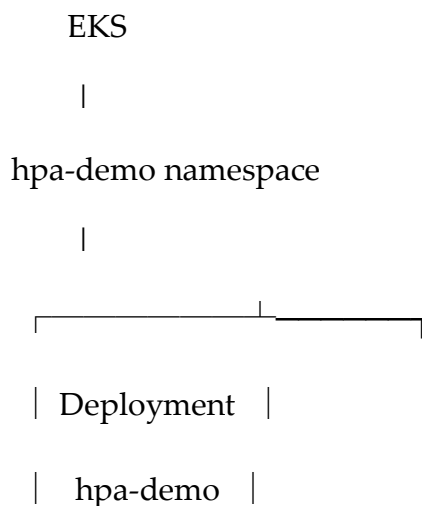
Then:

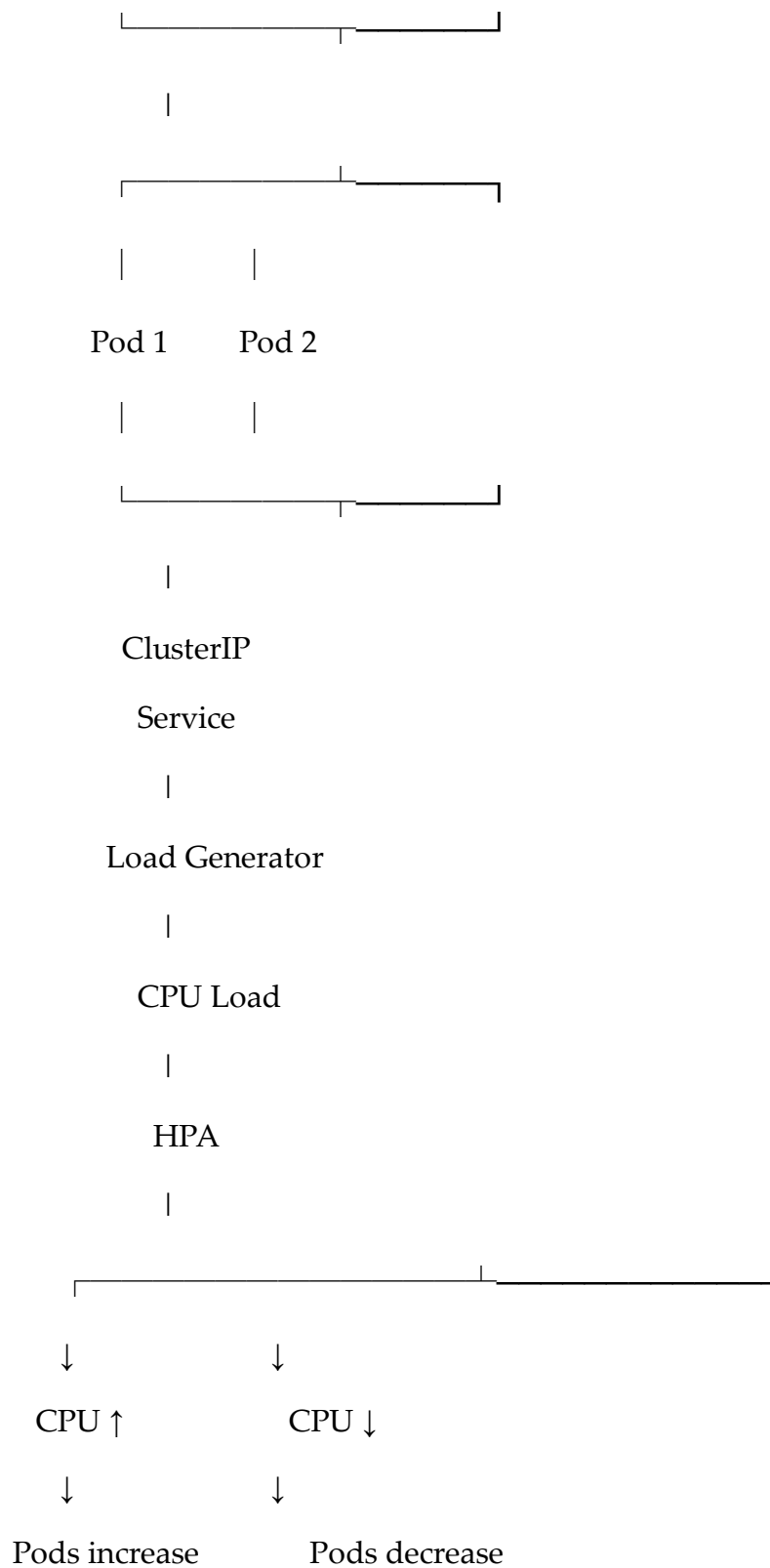
```
kubectl get hpa -n hpa-demo -w
```

After the HPA's scale-down stabilization/measurement behavior takes effect, the replicas should decrease toward:

2

Step 22 — Final Expected Architecture





HPA automatically increases or decreases the number of Pods managed by a workload such as a Deployment based on observed resource utilization or other

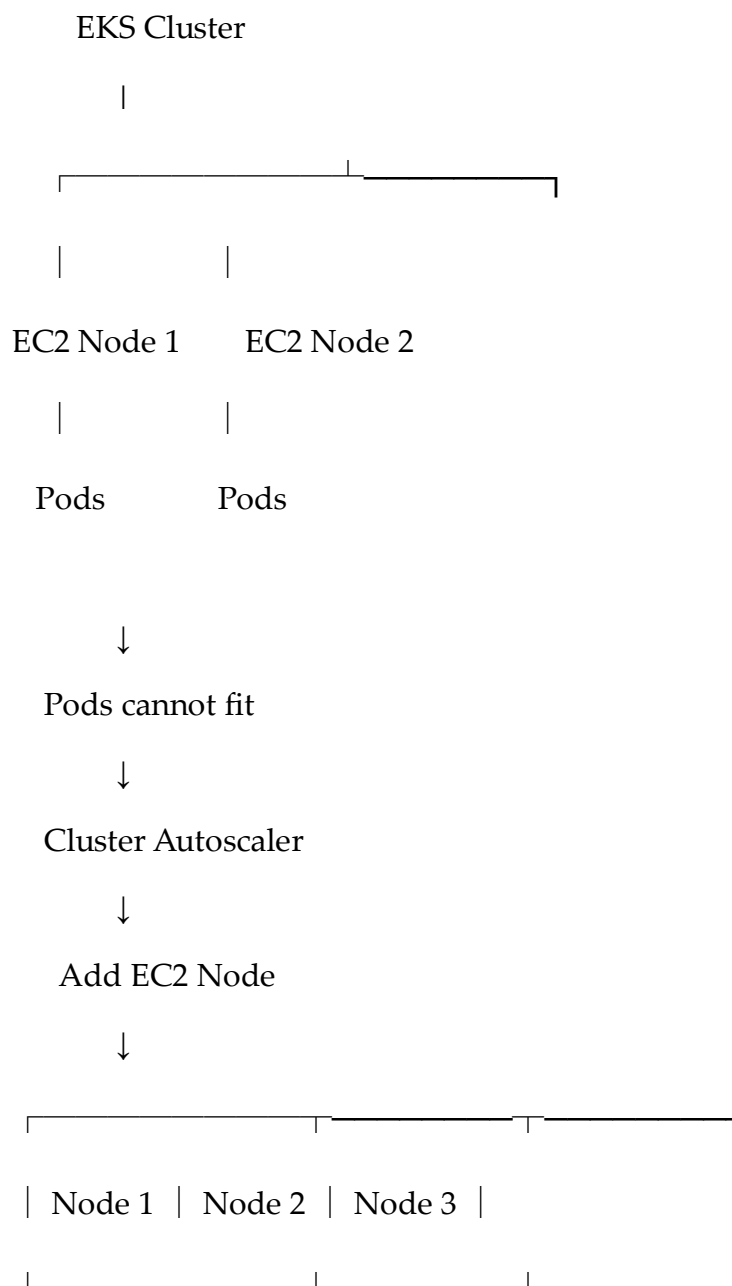
supported metrics, while maintaining the configured minimum and maximum replica limits.

4. Setup cluster autoscale

Setup Cluster Autoscaler on EKS

1. What are we building?

The goal is:



When workloads decrease:

Low workload



Unused node



Cluster Autoscaler



Remove node

So remember:

HPA

Cluster Autoscaler



Pods

EC2 Nodes

2 → 5 Pods

2 → 3 Nodes

2. Prerequisites

Before starting, verify your EKS cluster.

`kubectl get nodes`

You should see something like:

NAME	STATUS	ROLES
ip-10-0-1-xxx.ec2.internal	Ready	<none>
ip-10-0-2-xxx.ec2.internal	Ready	<none>

Also check:

`kubectl get pods -A`

Your system pods should be healthy.

3. Check Your EKS Node Group

This is very important.

Cluster Autoscaler needs to know:

Minimum number of nodes

Maximum number of nodes

Desired number of nodes

Go to:

AWS Console

→ EKS

→ Clusters

→ Your cluster

→ Compute

You should see your managed node group.

For example:

Node group:

console-node-group

Desired:

2

Minimum:

1

Maximum:

3

If your node group currently has:

Min = 1

Desired = 2

Max = 2

change **Maximum** to at least:

Max = 3

For this lab I recommend:

Minimum = 1

Desired = 2

Maximum = 3

Later we will make Kubernetes request more nodes.

4. Why Maximum Matters

Suppose you have:

2 EC2 nodes

and:

Maximum = 2

Cluster Autoscaler cannot create a third node.

So:

Current nodes = 2

Maximum nodes = 2

Pod pending

↓

Cluster Autoscaler wants node

↓

Cannot exceed max = 2

↓

Pod remains Pending

Therefore:

Minimum = 1

Maximum = 3

is useful for our lab.

5. Give Cluster Autoscaler Permission

Cluster Autoscaler needs AWS permissions to change the EKS node group's desired capacity.

There are two common approaches:

Option 1:

IAM Role for Service Account (IRSA)

Option 2:

EKS Pod Identity

For current EKS environments, **EKS Pod Identity** is a good approach.

However, if your training material specifically expects IRSA, use IRSA.

For learning the classic Cluster Autoscaler lab, I'll show the **IRSA approach**, because it helps you understand the IAM relationship.

6. Check OIDC Provider

Run:

```
aws eks describe-cluster \
  --name console-eks-cluster \
  --region us-east-1 \
  --query "cluster.identity.oidc.issuer" \
  --output text
```

You should get something similar to:

<https://oidc.eks.us-east-1.amazonaws.com/id/XXXXXXXXXXXX>

This is the OIDC identity provider associated with your EKS cluster.

7. Create IAM Policy

Go to:

AWS Console

→ IAM

→ Policies

→ Create policy

Choose:

JSON

Use a policy allowing Cluster Autoscaler to inspect and modify Auto Scaling groups.

For a lab, you can use:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "autoscaling:DescribeAutoScalingGroups",
        "autoscaling:DescribeAutoScalingInstances",
        "autoscaling:DescribeLaunchConfigurations",
        "autoscaling:DescribeScalingActivities",
        "autoscaling:DescribeTags",
        "ec2:DescribeImages",
        "ec2:DescribeInstanceTypes",
```

```
"ec2:DescribeLaunchTemplateVersions",
"autoscaling:SetDesiredCapacity",
"autoscaling:TerminateInstanceInAutoScalingGroup"
],
"Resource": "*"
}
]
```

Click:

Next

Name it:

EKSClusterAutoscalerPolicy

Then:

Create policy

For production, tighten IAM permissions further instead of using Resource: "*".

8. Create IAM Role for Cluster Autoscaler

Go to:

IAM

→ Roles

→ Create role

For trusted entity, select the web identity/OIDC option corresponding to your EKS cluster.

You want the role to be assumable by the Kubernetes service account:

system:serviceaccount:kube-system:cluster-autoscaler

Attach:

EKSClusterAutoscalerPolicy

Give the role a name:

EKSClusterAutoscalerRole

9. Important IAM Relationship

Understand this architecture:

Cluster Autoscaler Pod

|

| Kubernetes ServiceAccount

↓

cluster-autoscaler

|

| OIDC / IRSA

↓

IAM Role

|

↓

AWS Auto Scaling API

|

↓

EKS Managed Node Group

|

↓

EC2 Node

This is one of the most important concepts in this lab.

10. Add IAM Role Annotation

Once the role exists, we will associate it with the Kubernetes service account.

The annotation will look conceptually like:

```
apiVersion: v1
```

```
kind: ServiceAccount
```

```
metadata:
```

```
  name: cluster-autoscaler
```

```
  namespace: kube-system
```

```
  annotations:
```

```
    eks.amazonaws.com/role-arn:
```

```
arn:aws:iam::<ACCOUNT_ID>:role/EKSClusterAutoscalerRole
```

Replace:

```
<ACCOUNT_ID>
```

with your AWS account ID.

You can get it with:

```
aws sts get-caller-identity
```

11. Create Service Account

Create:

```
cluster-autoscaler-sa.yaml
```

Use:

```
apiVersion: v1
```

```
kind: ServiceAccount
```

```
metadata:
```

```
  name: cluster-autoscaler
```

```
  namespace: kube-system
```

annotations:

eks.amazonaws.com/role-arn:

arn:aws:iam::ACCOUNT_ID:role/EKSClusterAutoscalerRole

Replace:

ACCOUNT_ID

with your account ID.

Apply:

kubectl apply -f cluster-autoscaler-sa.yaml

Check:

kubectl get serviceaccount cluster-autoscaler -n kube-system

12. Install Cluster Autoscaler

Now deploy Cluster Autoscaler.

There is an important point here:

Do not blindly use an old Cluster Autoscaler image/version from an old tutorial.

Your Cluster Autoscaler version should be compatible with your Kubernetes/EKS version.

First check your Kubernetes version:

kubectl version

Also:

kubectl get nodes

Then use the Cluster Autoscaler release corresponding to your Kubernetes minor version.

For example, if your cluster is:

v1.34.x

use a Cluster Autoscaler release intended for Kubernetes 1.34.

13. Download the Cluster Autoscaler Manifest

For the matching release, use the official Cluster Autoscaler deployment manifest.

The important components are:

Deployment

ServiceAccount

RBAC permissions

ClusterRole

ClusterRoleBinding

The Deployment will run:

cluster-autoscaler

inside:

kube-system

14. Add the Node Group Configuration

The Cluster Autoscaler needs to know which node groups it can scale.

For a classic managed node group, the arguments look like:

```
--nodes=1:3:<node-group-auto-scaling-group>
```

Conceptually:

```
--nodes=<MIN>:<MAX>:<NODE_GROUP>
```

For example:

```
--nodes=1:3:console-node-group
```

However, with EKS managed node groups, it's generally better to use AWS auto-discovery tags rather than hard-coding the ASG.

15. Recommended Auto-Discovery Configuration

Cluster Autoscaler can discover node groups using tags.

The important tags are:

k8s.io/cluster-autoscaler/enabled = true

k8s.io/cluster-autoscaler/<CLUSTER_NAME> = owned

For your cluster:

k8s.io/cluster-autoscaler/enabled=true

k8s.io/cluster-autoscaler/console-eks-cluster=owned

16. Add Tags to the Node Group

Go to:

AWS Console

→ EC2

→ Auto Scaling Groups

Find the Auto Scaling Group associated with your EKS managed node group.

Open:

Tags

Add:

Key:

k8s.io/cluster-autoscaler/enabled

Value:

true

And:

Key:

k8s.io/cluster-autoscaler/console-eks-cluster

Value:

owned

Make sure:

Propagate to instances

is enabled where applicable.

17. Why We Add These Tags

Cluster Autoscaler searches AWS Auto Scaling Groups.

It finds:

ASG

|

+-- cluster-autoscaler/enabled=true

|

+-- cluster-autoscaler/console-eks-cluster=owned

Then:

Cluster Autoscaler

↓

Find matching ASG

↓

Check min/max

↓

Change desired capacity

18. Deploy Cluster Autoscaler

Your Deployment should contain arguments similar to:

containers:

- name: cluster-autoscaler

image: registry.k8s.io/autoscaling/cluster-autoscaler:<MATCHING_VERSION>

command:

- ./cluster-autoscaler

- --cloud-provider=aws

- --cluster-name=console-eks-cluster

- --balance-similar-node-groups

- --skip-nodes-with-system-pods=false

- --expander=least-waste

- --node-group-auto-discovery=asg:tag=k8s.io/cluster-autoscaler/enabled,k8s.io/cluster-autoscaler/console-eks-cluster

The exact manifest should come from the Cluster Autoscaler release that matches your Kubernetes version.

19. Check Cluster Autoscaler Pod

Run:

```
kubectl get pods -n kube-system | grep autoscaler
```

You want:

```
cluster-autoscaler-xxxxx 1/1 Running
```

If you see:

```
CrashLoopBackOff
```

or:

```
Error
```

don't continue to the scaling test yet.

20. Check Logs

Run:

```
kubectl logs -n kube-system deployment/cluster-autoscaler
```

You want to see messages indicating that Cluster Autoscaler has discovered your node group.

For example:

Regenerating instance to ASG map

or messages indicating the node group and its min/max sizes.

21. The Most Important Test

Now we'll deliberately create more Pods than your current nodes can handle.

Suppose your current node group has:

Min = 1

Desired = 2

Max = 3

We'll create Pods that require enough resources to make some Pods remain Pending.

22. Create Test Deployment

Create:

cluster-autoscaler-test.yaml

Use:

apiVersion: apps/v1

kind: Deployment

metadata:

name: autoscaler-test

namespace: default

spec:

replicas: 2

```
selector:
  matchLabels:
    app: autoscaler-test
template:
  metadata:
    labels:
      app: autoscaler-test
spec:
  containers:
  - name: nginx
    image: nginx:latest
    resources:
      requests:
        cpu: "1500m"
        memory: "1Gi"
      limits:
        cpu: "1500m"
        memory: "1Gi"
```

Apply:

```
kubectl apply -f cluster-autoscaler-test.yaml
```

23. Check Pods

Run:

```
kubectl get pods -o wide
```

You might see:

```
autoscaler-test-xxxxx  Running
```

autoscaler-test-yyyyy Pending

This is exactly what we want to test.

Why?

Because there isn't enough capacity.

24. Check Why Pod Is Pending

Run:

```
kubectl describe pod <POD_NAME>
```

At the bottom, check:

Events:

You may see:

0/2 nodes are available:

Insufficient cpu

This means:

Pod needs CPU

↓

Current nodes don't have enough

↓

Pod stays Pending

25. Cluster Autoscaler Detects Pending Pod

Now Cluster Autoscaler sees:

Pending Pod

↓

No existing node can schedule it

↓

Can adding a node help?

↓

YES

↓

Increase ASG desired capacity

Then:

AWS Auto Scaling Group

↓

Desired: 2 → 3

↓

EC2 launches

↓

New Kubernetes node joins

↓

Pending Pod scheduled

26. Watch Nodes

Open another terminal:

```
kubectl get nodes -w
```

Initially:

2 nodes

After scaling:

3 nodes

You should eventually see:

NAME	STATUS
ip-10-0-1-xxx	Ready
ip-10-0-2-xxx	Ready

ip-10-0-3-xxx Ready

27. Watch Pods

Run:

```
kubectl get pods -o wide -w
```

Initially:

Pod 1 Running

Pod 2 Pending

After the new node joins:

Pod 1 Running

Pod 2 Running

 Cluster Autoscaler has worked.

28. Verify AWS Console

Go to:

AWS Console

→ EC2

→ Instances

You should see the additional worker node.

For example:

Before:

EC2

└── Worker 1

└── Worker 2

After:

EC2

└── Worker 1

└── Worker 2

└── Worker 3

Also check:

EKS

→ Cluster

→ Compute

→ Node group

You should see:

Desired:

3

instead of:

Desired:

2

29. Now Test Scale Down

Delete the test workload:

```
kubectl delete deployment autoscaler-test
```

Check:

```
kubectl get pods
```

Your extra workload disappears.

The third node becomes unnecessary.

Cluster Autoscaler eventually detects:

Node no longer needed

and scales down:

3 nodes

↓

2 nodes

Remember that scale-down is **not immediate**.

Cluster Autoscaler waits and evaluates whether a node can safely be removed.

30. Watch Scale Down

Run:

```
kubectl get nodes -w
```

You should eventually return to:

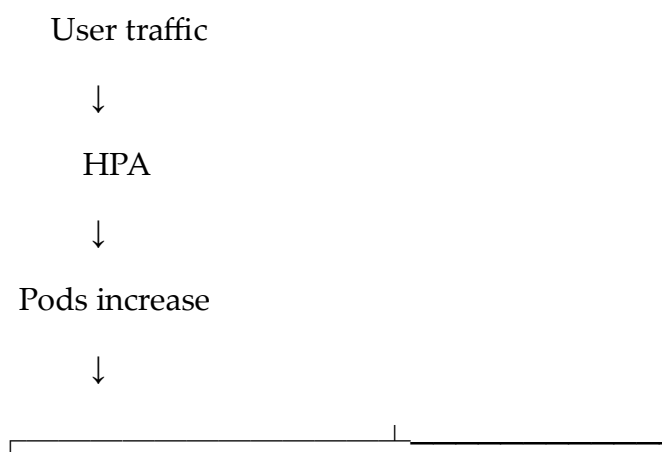
2 nodes

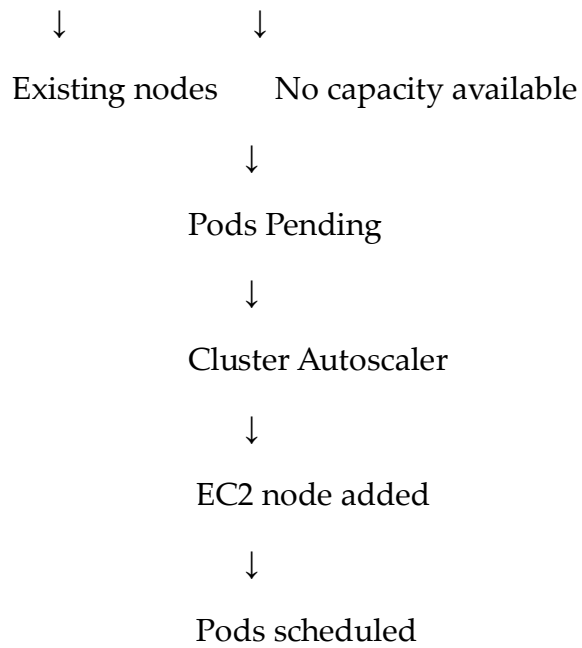
The exact timing depends on Cluster Autoscaler configuration and its scale-down behavior.

31. Very Important: HPA + Cluster Autoscaler

Now connect this to your previous HPA lab.

Imagine:





This is the relationship you should remember:

HPA



More Pods

Cluster Autoscaler



More Nodes

32. HPA + Cluster Autoscaler Production Example

Suppose you start with:

Nodes = 2

Pods = 4

Traffic increases:

CPU increases



HPA



4 Pods → 8 Pods

But the existing nodes cannot accommodate all 8 Pods.

Some become:

Pending

Cluster Autoscaler detects that:

Pending Pods



Need more capacity



2 nodes → 3 nodes

Then:

Pods



Pending → Running

So:

Traffic



HPA



More Pods



Insufficient Node Capacity



Cluster Autoscaler



More EC2 Nodes

↓

Pods scheduled

33. Important Difference

Component	Scales	Example
HPA	Pods	2 → 6 Pods
Cluster Autoscaler	Nodes	2 → 3 EC2 nodes
VPA	Pod resources	100m → 500m CPU
Karpenter	Node capacity	Launches right-sized nodes

34. Troubleshooting

Problem 1 — Pod remains Pending

Run:

```
kubectl describe pod <pod-name>
```

Look at:

Events

If you see:

Insufficient cpu

that's useful for our test.

Then check:

```
kubectl logs -n kube-system deployment/cluster-autoscaler
```

Problem 2 — Cluster Autoscaler Pod isn't Running

Run:

```
kubectl get pods -n kube-system | grep autoscaler
```

Then:

```
kubectl describe pod <autoscaler-pod> -n kube-system
```

And:

```
kubectl logs <autoscaler-pod> -n kube-system
```

Problem 3 — IAM AccessDenied

If logs contain something like:

AccessDenied

check:

ServiceAccount

↓

IAM role annotation

↓

Trust policy

↓

IAM permissions

Verify:

```
kubectl get serviceaccount cluster-autoscaler \
```

```
-n kube-system \
```

```
-o yaml
```

Look for:

[eks.amazonaws.com/role-arn:](https://eks.amazonaws.com/role-arn)

Problem 4 — Node group doesn't scale

Check the ASG tags.

You need:

k8s.io/cluster-autoscaler/enabled=true

and:

k8s.io/cluster-autoscaler/console-eks-cluster=owned

Also check:

EKS

→ Cluster

→ Compute

→ Node group

and confirm:

Minimum < Maximum

For our lab:

Minimum = 1

Maximum = 3

35. Useful Commands

Nodes

kubectl get nodes

Nodes with resources

kubectl get nodes -o wide

Pods

kubectl get pods -A

Pending Pods

kubectl get pods --field-selector=status.phase=Pending

Autoscaler

kubectl get pods -n kube-system | grep autoscaler

Autoscaler logs

```
kubectl logs -n kube-system deployment/cluster-autoscaler
```

Autoscaler description

```
kubectl describe deployment cluster-autoscaler -n kube-system
```

Node details

```
kubectl describe node <node-name>
```

Check resource usage

```
kubectl top nodes
```

```
kubectl top pods -A
```

36. Complete Lab Flow

Your complete hands-on flow is:

1. EKS Cluster

↓

2. Managed Node Group

↓

3. Set:

Min = 1

Desired = 2

Max = 3

↓

4. IAM Policy

↓

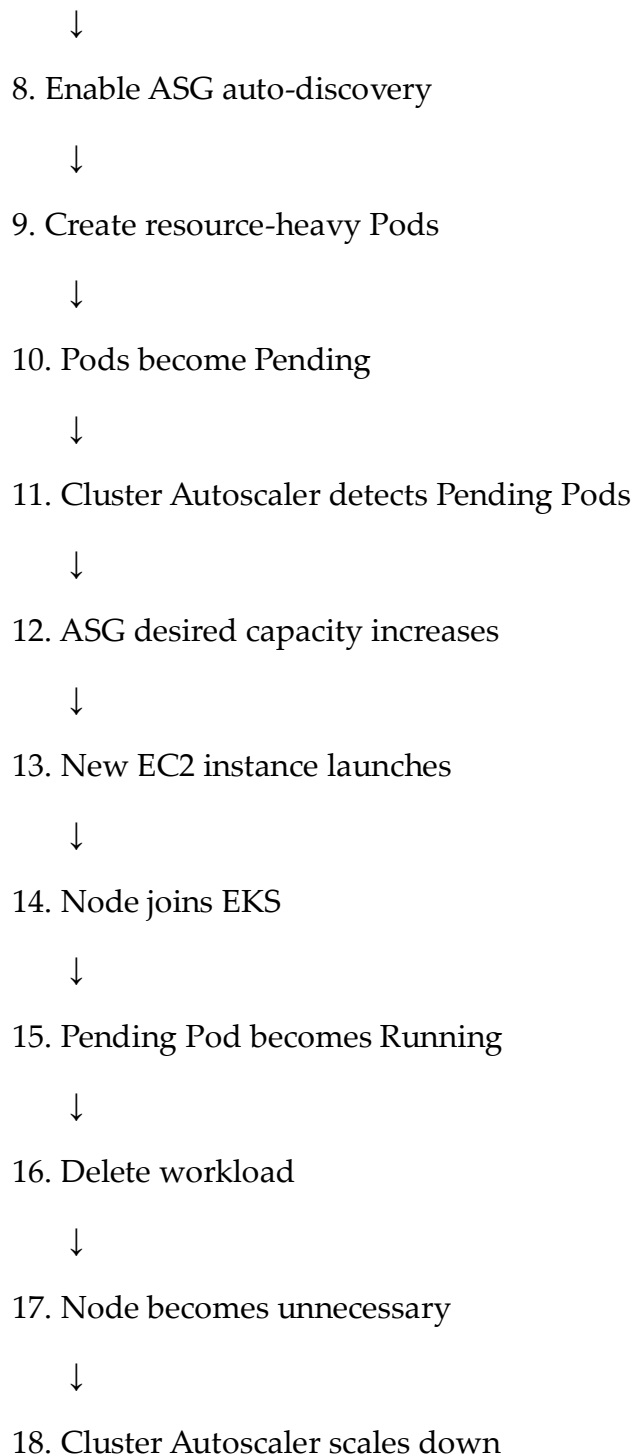
5. IAM Role

↓

6. ServiceAccount

↓

7. Cluster Autoscaler



Setup Kubernetes Job and CronJob on EKS

1. First understand the difference

Job

A **Job** runs a task and makes sure it completes successfully.

Job



Pod



Task



Completed

Example:

Backup database

Generate report

Run migration

Process a file

A Job normally runs **once**.

CronJob

A **CronJob** runs Jobs according to a schedule.

CronJob



| Every 5 minutes



Job



Pod



Completed

For example:

Every day at 2 AM

↓

CronJob

↓

Job

↓

Backup Pod

Part A — Create a Job

2. Verify your EKS cluster

First:

```
kubectl get nodes
```

You should see:

NAME	STATUS	ROLES
ip-10-0-x-x.ec2.internal	Ready	<none>
ip-10-0-x-x.ec2.internal	Ready	<none>

All nodes should be:

Ready

3. Create a Namespace

Let's create a separate namespace for this lab:

```
kubectl create namespace jobs-demo
```

Verify:

```
kubectl get namespaces
```

You should see:

jobs-demo

4. Create a Job YAML

Create a file:

job.yaml

Put this inside:

apiVersion: batch/v1

kind: Job

metadata:

name: simple-job

namespace: jobs-demo

spec:

completions: 1

backoffLimit: 3

template:

spec:

restartPolicy: Never

containers:

- name: job-container

image: busybox:1.36

command:

- /bin/sh

- -c

- |

echo "Job started"

echo "Processing data..."

sleep 10

echo "Job completed successfully"

5. Understand the YAML

Let's understand each important field.

Job

kind: Job

This tells Kubernetes:

Create a Kubernetes Job.

Completions

completions: 1

Means:

1 successful Pod completion

Backoff Limit

backoffLimit: 3

If the Pod fails, Kubernetes can retry the Job.

Conceptually:

Attempt 1 → Failed

Attempt 2 → Failed

Attempt 3 → Failed

...

The Job eventually reaches failure according to the configured retry behavior.

Restart Policy

restartPolicy: Never

The Pod itself won't restart its container after termination.

The **Job controller** handles creating replacement Pods when appropriate.

6. Create the Job

Run:

```
kubectl apply -f job.yaml
```

Expected:

```
job.batch/simple-job created
```

7. Check the Job

Run:

```
kubectl get jobs -n jobs-demo
```

Initially you might see:

NAME	COMPLETIONS	DURATION	AGE
simple-job	0/1	2s	2s

After completion:

NAME	COMPLETIONS	DURATION	AGE
simple-job	1/1	10s	20s

The important part is:

```
1/1
```

That means the Job successfully completed.

8. Check the Pod

Run:

```
kubectl get pods -n jobs-demo
```

You may see:

NAME	READY	STATUS	RESTARTS
simple-job-xxxxxx	0/1	Completed	0

This is **normal**.

A Job Pod isn't expected to remain Running.

It should become:

Completed

9. Check Job Logs

Run:

```
kubectl logs -n jobs-demo job/simple-job
```

You should see:

Job started

Processing data...

Job completed successfully

This is a very useful command when troubleshooting Jobs.

10. Describe the Job

Run:

```
kubectl describe job simple-job -n jobs-demo
```

At the bottom you'll see events and information such as:

Pods Statuses:

1 Active

1 Succeeded

After completion, typically:

1 Succeeded

11. Check Job YAML

You can also inspect it:

```
kubectl get job simple-job -n jobs-demo -o yaml
```

Look for:

status:

 succeeded: 1

That means:

Job successful

Part B — Create a CronJob

Now let's create a CronJob.

12. What Does CronJob Do?

A CronJob does **not directly run a Pod**.

The relationship is:

CronJob

↓

creates Job

↓

Job creates Pod

↓

Pod runs task

↓

Pod completes

For example:

CronJob

Schedule: every 5 minutes

10:00



Job 1



Pod 1



Completed

10:05



Job 2



Pod 2



Completed

10:10



Job 3



Pod 3



Completed

13. Create CronJob YAML

Create:

cronjob.yaml

Use:

apiVersion: batch/v1

kind: CronJob

metadata:

name: hello-cronjob

namespace: jobs-demo

spec:

schedule: "*/5 * * * *"

successfulJobsHistoryLimit: 3

failedJobsHistoryLimit: 1

jobTemplate:

spec:

backoffLimit: 2

template:

spec:

restartPolicy: Never

containers:

- name: cronjob-container

image: busybox:1.36

command:

- /bin/sh

- -c

- |

```
echo "CronJob started"

echo "Current date and time:"

date

echo "CronJob completed"
```

14. Understand the Cron Schedule

This:

```
*/5 * * * *
```

means:

Every 5 minutes

The five fields are:

└──────── minute

| └──────── hour

| | └──────── day of month

| | | └── month

| | | | └ day of week

| | | | |

```
* * * * *
```

15. Common Cron Schedules

Every minute

```
* * * * *
```

Every 5 minutes

```
*/5 * * * *
```

Every hour

0 * * * *

Every day at midnight

0 0 * * *

Every day at 2 AM

0 2 * * *

Every Sunday at 2 AM

0 2 * * 0

16. Apply the CronJob

Run:

```
kubectl apply -f cronjob.yaml
```

Expected:

```
cronjob.batch/hello-cronjob created
```

17. Check CronJob

Run:

```
kubectl get cronjobs -n jobs-demo
```

You should see:

NAME	SCHEDULE	SUSPEND	ACTIVE	LAST SCHEDULE
hello-cronjob	*/* * * * *	False	0	<none>

After the first scheduled execution:

NAME	SCHEDULE	SUSPEND	ACTIVE	LAST SCHEDULE
hello-cronjob	*/* * * * *	False	0	2m

18. Check Created Jobs

Run:

```
kubectl get jobs -n jobs-demo
```

Initially you already have:

```
simple-job
```

Then you'll see something like:

NAME	COMPLETIONS	DURATION
simple-job	1/1	10s
hello-cronjob-12345678	1/1	2s

The exact generated Job name will be different.

19. Check CronJob Pods

Run:

```
kubectl get pods -n jobs-demo
```

You may see:

NAME	STATUS
simple-job-xxxxx	Completed
hello-cronjob-12345678-xxxxx	Completed

After another 5 minutes, another CronJob-created Pod will appear.

20. Check CronJob Logs

First get the Job:

```
kubectl get jobs -n jobs-demo
```

Then:

```
kubectl logs -n jobs-demo job/<cronjob-job-name>
```

For example:

```
kubectl logs -n jobs-demo job/hello-cronjob-12345678
```

You should see:

CronJob started

Current date and time:

Mon Oct 5 12:30:00 UTC 2026

CronJob completed

The exact time will depend on your cluster/node environment.

21. Important: Kubernetes CronJob Uses Cluster Time

CronJob schedules are evaluated according to the Kubernetes CronJob controller's configured time behavior.

If your expected local time and observed execution time differ, check the cluster/controller timezone configuration and use an explicit `timeZone` field when supported by your Kubernetes version.

For example:

spec:

```
schedule: "0 9 * * *"
```

```
timeZone: "Asia/Kolkata"
```

That means:

Every day at 9:00 AM

India Standard Time

For your current EKS lab, you can use:

```
timeZone: "Asia/Kolkata"
```

if your EKS/Kubernetes version supports it.

22. Test CronJob Immediately

Waiting 5 minutes every time isn't convenient for a lab.

You can manually create a Job from the CronJob.

Run:

```
kubectl create job --from=cronjob/hello-cronjob manual-cron-test -n jobs-demo
```

Expected:

```
job.batch/manual-cron-test created
```

Check:

```
kubectl get jobs -n jobs-demo
```

You should see:

```
manual-cron-test 1/1
```

Then:

```
kubectl logs -n jobs-demo job/manual-cron-test
```

23. Job vs CronJob

Remember this table:

Feature	Job	CronJob
Runs	Once	On schedule
Creates Pod	Yes	Through Job
Scheduling	No	Yes
Example	Database migration	Daily backup
Completion	Yes	Each created Job completes
API	batch/v1	batch/v1

24. Production Example

Imagine you have a production application:

E-commerce Application

Every night at 2 AM you want to back up some data.

Use:

CronJob



Job



Backup Pod



Backup database



Store backup in S3



Completed

Another example:

CronJob

schedule: 0 2 * * *

means:

Every day at 2 AM

25. Important Production Settings

For production CronJobs, consider:

concurrencyPolicy

Example:

concurrencyPolicy: Forbid

Meaning:

If previous Job is still running



Don't start another one

This is useful for backups.

successfulJobsHistoryLimit

Example:

successfulJobsHistoryLimit: 3

Keep only the latest 3 successful Jobs.

failedJobsHistoryLimit

Example:

failedJobsHistoryLimit: 1

Keep only the latest failed Job.

startingDeadlineSeconds

Controls how long Kubernetes can try to start a missed scheduled Job.

26. Production CronJob Example

A more realistic example:

apiVersion: batch/v1

kind: CronJob

metadata:

name: daily-backup

namespace: jobs-demo

spec:

schedule: "0 2 * * *"

timeZone: "Asia/Kolkata"

concurrencyPolicy: Forbid

successfulJobsHistoryLimit: 3

failedJobsHistoryLimit: 1

jobTemplate:

spec:

backoffLimit: 3

template:

spec:

restartPolicy: Never

containers:

- name: backup

image: busybox:1.36

command:

- /bin/sh

- -c

- |

echo "Starting backup..."

date

echo "Backup completed"

This is the type of configuration you would build on top of once the basic lab works.

27. Troubleshooting

Problem 1 — Job is Pending

Run:

kubectl get pods -n jobs-demo

If:

STATUS

Pending

run:

```
kubectl describe pod <pod-name> -n jobs-demo
```

Check:

Events:

Common reasons:

Insufficient CPU

Insufficient memory

No suitable node

Taints

NodeSelector mismatch

PVC problems

28. Problem 2 — Job Failed

Run:

```
kubectl get jobs -n jobs-demo
```

Then:

```
kubectl describe job simple-job -n jobs-demo
```

Get the Pod:

```
kubectl get pods -n jobs-demo
```

Then:

```
kubectl logs <pod-name> -n jobs-demo
```

29. Problem 3 — CronJob Isn't Creating Jobs

Run:

```
kubectl get cronjob -n jobs-demo
```

Check:

SCHEDULE

SUSPEND

LAST SCHEDULE

Then:

kubectl describe cronjob hello-cronjob -n jobs-demo

Check the events.

Also verify that:

SUSPEND = False

30. Problem 4 — CronJob Creates Too Many Jobs

Check:

successfulJobsHistoryLimit: 3

failedJobsHistoryLimit: 1

And consider:

concurrencyPolicy: Forbid

This helps prevent overlapping executions.

31. Complete Lab Verification

Run these commands:

kubectl get jobs -n jobs-demo

kubectl get cronjobs -n jobs-demo

kubectl get pods -n jobs-demo

kubectl logs -n jobs-demo job/simple-job

kubectl describe cronjob hello-cronjob -n jobs-demo

You should have something like:

CronJob

|

|— Job 1 → Pod → Completed

|

|— Job 2 → Pod → Completed

|

|— Job 3 → Pod → Completed

32. Cleanup

When you're finished:

`kubectl delete namespace jobs-demo`

This removes:

Job

CronJob

Pods

Verify:

`kubectl get namespace jobs-demo`

You should eventually get:

Error from server (NotFound)

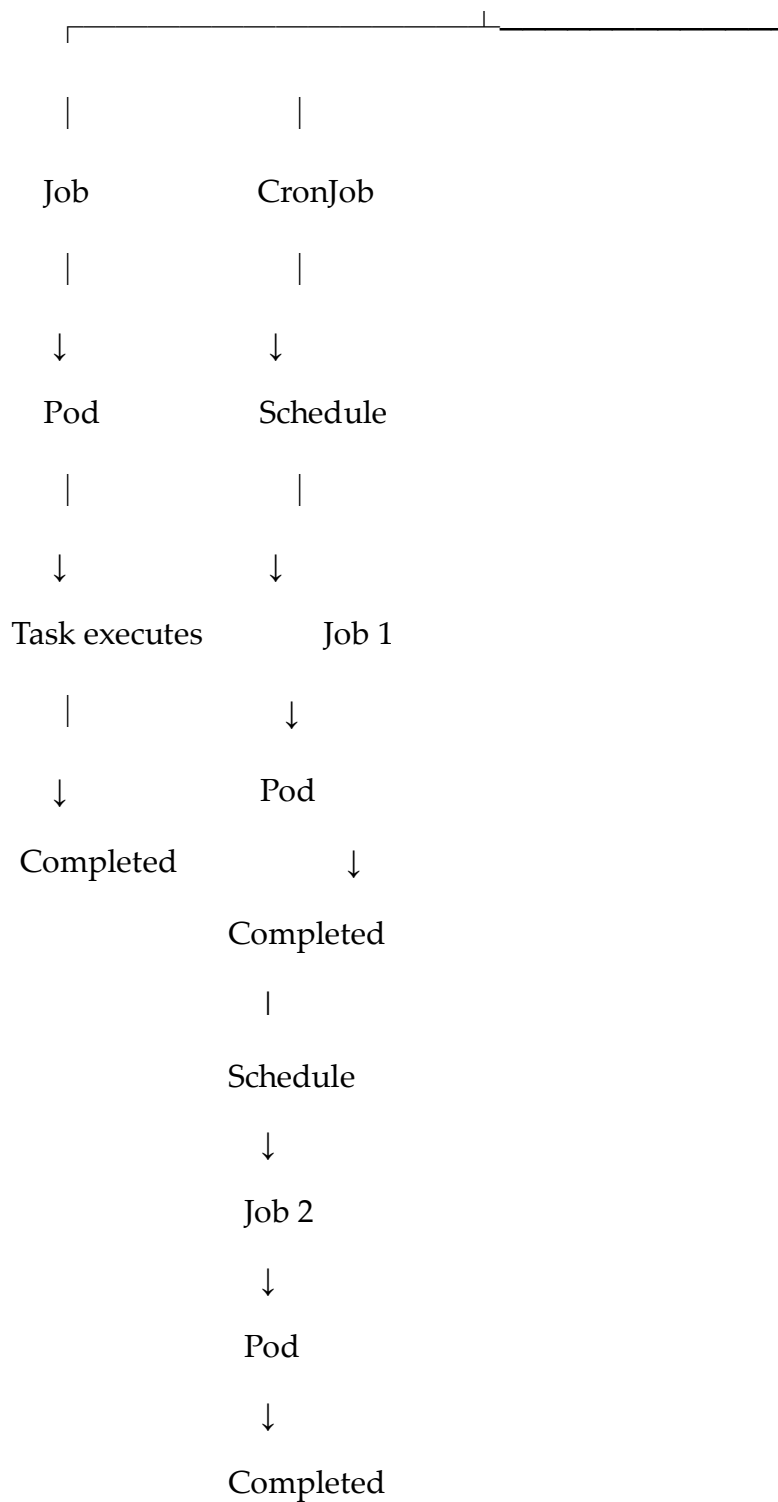
33. Your Complete Hands-on Flow

EKS Cluster

|

jobs-demo namespace

|



Interview answer

Job:

A Kubernetes Job creates one or more Pods and ensures that a specified task successfully completes.

CronJob:

A Kubernetes CronJob creates Jobs on a recurring schedule, similar to Linux cron.

The most important relationship to remember is:

Job:

Job → Pod → Task → Completed

CronJob:

CronJob → Job → Pod → Task → Completed

And one important point for your EKS lab: **Job/CronJob are Kubernetes workload resources; they do not automatically create EC2 nodes.** If a Job creates a Pod that cannot be scheduled because the cluster has insufficient capacity, your **Cluster Autoscaler** can potentially add a node, just like in the previous lab.

6. Create secret and inject inside pod

Create Secret and Inject Inside Pod

Objective

We will learn how to:

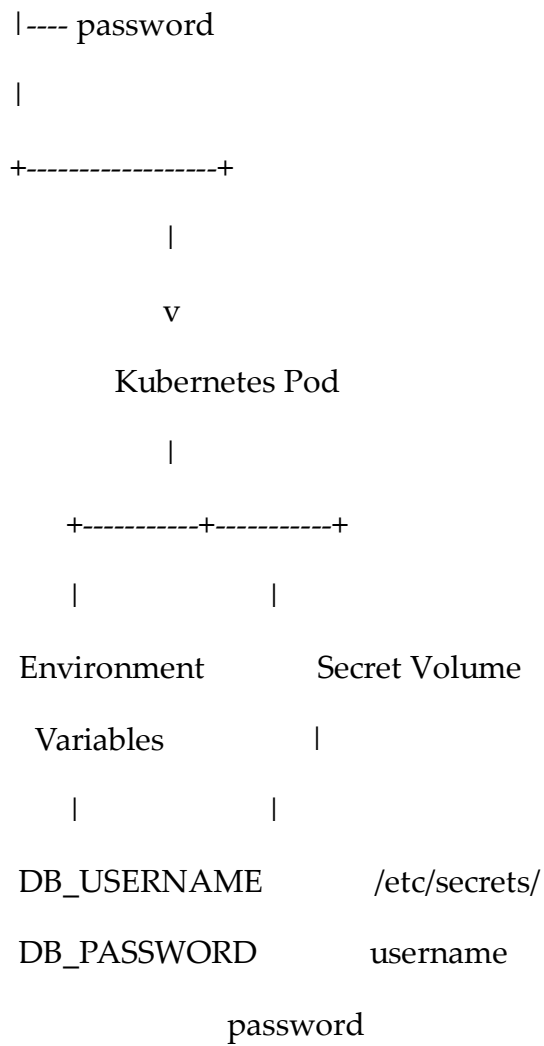
1. Create a Kubernetes Secret
2. Store username/password in the Secret
3. Inject Secret values into a Pod as **environment variables**
4. Inject Secret values as **files**
5. Verify the Secret inside the Pod
6. Understand how Secrets are used in production

Architecture

Kubernetes Secret

|

|---- username



Step 1: Check your EKS cluster

Run:

```
kubectl get nodes
```

Expected:

NAME	STATUS	ROLES	AGE	VERSION
ip-10-0-8-xxx.ec2.internal	Ready	<none>	...	v1.34.x
ip-10-0-9-xxx.ec2.internal	Ready	<none>	...	v1.34.x

Also check the current context:

```
kubectl config current-context
```

Make sure you are connected to your **EKS cluster**, not your old kubeadm/minikube cluster.

Step 2: Create a namespace

Let's create a separate namespace for this lab.

```
kubectl create namespace secret-demo
```

Verify:

```
kubectl get namespace secret-demo
```

Step 3: Create a Secret

There are several ways to create a Secret.

For beginners, let's use the command line first.

Create username:

```
kubectl create secret generic app-secret \  
  --namespace=secret-demo \  
  --from-literal=username=admin \  
  --from-literal=password='MyP@ss123'
```

Expected:

```
secret/app-secret created
```

Step 4: Verify the Secret

Run:

```
kubectl get secrets -n secret-demo
```

Expected:

NAME	TYPE	DATA	AGE
app-secret	Opaque	2	10s

Notice:

DATA = 2

because we stored:

username

password

Step 5: Describe the Secret

Run:

```
kubectl describe secret app-secret -n secret-demo
```

You should see something similar to:

Name: app-secret

Namespace: secret-demo

Type: Opaque

Data

====

password: 10 bytes

username: 5 bytes

Important

Kubernetes does **not** display the actual secret values in `kubectl describe`.

That is intentional.

Step 6: Understand how Secret data is stored

Run:

```
kubectl get secret app-secret -n secret-demo -o yaml
```

You will see something like:

apiVersion: v1

kind: Secret

metadata:

name: app-secret

type: Opaque

data:

password: TXlQQHNzMTIz

username: YWRtaW4=

The values are **Base64 encoded**.

For example:

admin

becomes:

YWRtaW4=

And:

MyP@ss123

becomes:

TXlQQHNzMTIz

⚠ Important interview point

Base64 is NOT encryption.

Anyone who has permission to read the Secret can decode it.

For example:

```
echo 'YWRtaW4=' | base64 -d
```

Output:

admin

So don't say:

Kubernetes Secret encrypts the password using Base64.

That's incorrect.

Say:

Kubernetes Secret stores sensitive data in a Secret object, with the manifest representation encoded using Base64. Base64 itself is not encryption.

Step 7: Inject Secret into a Pod as Environment Variables

Now create:

```
vi secret-env-pod.yaml
```

Add:

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: secret-env-pod
```

```
  namespace: secret-demo
```

```
spec:
```

```
  containers:
```

```
  - name: app
```

```
    image: busybox:1.36
```

```
    command:
```

```
    - /bin/sh
```

```
    - -c
```

```
    - |
```

```
      echo "Application started"
```

```
      echo "Username: $DB_USERNAME"
```

```
      echo "Password: $DB_PASSWORD"
```

```
      sleep 3600
```

env:

- name: DB_USERNAME

valueFrom:

secretKeyRef:

name: app-secret

key: username

- name: DB_PASSWORD

valueFrom:

secretKeyRef:

name: app-secret

key: password

Save and exit.

Step 8: Create the Pod

kubectl apply -f secret-env-pod.yaml

Expected:

pod/secret-env-pod created

Check:

kubectl get pods -n secret-demo

Expected:

NAME	READY	STATUS	RESTARTS	AGE
secret-env-pod	1/1	Running	0	10s

Step 9: Check the Environment Variables

Run:

kubectl exec -n secret-demo secret-env-pod -- env | grep DB_

Expected:

DB_USERNAME=admin

DB_PASSWORD=MyP@ss123

🔗 This proves that the Secret was successfully injected into the Pod.

Step 10: Check individual values

Username:

```
kubectl exec -n secret-demo secret-env-pod -- sh -c 'echo $DB_USERNAME'
```

Output:

admin

Password:

```
kubectl exec -n secret-demo secret-env-pod -- sh -c 'echo $DB_PASSWORD'
```

Output:

MyP@ss123

Step 11: Inject Secret as Files

Environment variables are one method.

Another very common method is to mount the Secret as a **volume**.

Create:

```
vi secret-volume-pod.yaml
```

Use:

```
apiVersion: v1
```

```
kind: Pod
```

```
metadata:
```

```
  name: secret-volume-pod
```

```
  namespace: secret-demo
```

spec:

containers:

- name: app

image: busybox:1.36

command:

- /bin/sh

- -c

- |

echo "Application started"

ls -l /etc/app-secret

sleep 3600

volumeMounts:

- name: secret-volume

mountPath: /etc/app-secret

readOnly: true

volumes:

- name: secret-volume

secret:

secretName: app-secret

Apply:

kubectl apply -f secret-volume-pod.yaml

Check:

kubectl get pods -n secret-demo

Expected:

NAME	READY	STATUS	RESTARTS	AGE
------	-------	--------	----------	-----

```
secret-env-pod    1/1    Running 0    ...
secret-volume-pod 1/1    Running 0    ...
```

Step 12: Check Secret files

Run:

```
kubectl exec -n secret-demo secret-volume-pod -- ls -l /etc/app-secret
```

Expected:

password

username

The Secret has become files:

/etc/app-secret/

├── username

└── password

Step 13: Read the username file

```
kubectl exec -n secret-demo secret-volume-pod -- cat /etc/app-secret/username
```

Output:

admin

Read password:

```
kubectl exec -n secret-demo secret-volume-pod -- cat /etc/app-secret/password
```

Output:

MyP@ss123

Environment Variable vs Secret Volume

You should understand this difference very clearly.

Method	Example	Where Secret appears
Environment variable	\$DB_PASSWORD	Process environment
Volume mount	/etc/app-secret/password	File inside container

Environment variable

env:

- name: DB_PASSWORD

valueFrom:

secretKeyRef:

name: app-secret

key: password

Application sees:

DB_PASSWORD=MyP@ss123

Volume

volumes:

- name: secret-volume

secret:

secretName: app-secret

Application sees:

/etc/app-secret/password

Step 14: Inject the entire Secret

You can also inject **all Secret keys** as environment variables.

Example:

apiVersion: v1

kind: Pod

metadata:

name: secret-envfrom-pod

namespace: secret-demo

spec:

containers:

- name: app

image: busybox:1.36

command:

- /bin/sh

- -c

- |

env

sleep 3600

envFrom:

- secretRef:

name: app-secret

Apply:

kubectl apply -f secret-envfrom-pod.yaml

Then:

kubectl exec -n secret-demo secret-envfrom-pod -- env | grep -E
'username|password'

The exact environment variable names will correspond to the Secret keys.

Step 15: Production Example — Database Password

This is where Secrets are commonly used.

Imagine your application needs:

Database:

Host = mysql.example.com

Username = appuser

Password = MySecretPassword

You don't want this inside the Docker image.

✗ Bad:

env:

- name: DB_PASSWORD

value: MySecretPassword

Instead:

Kubernetes Secret

|

| DB_PASSWORD

v

Application Pod

|

v

Database

The Pod can reference:

env:

- name: DB_PASSWORD

valueFrom:

secretKeyRef:

name: database-secret

key: password

Step 16: Secret + Deployment

In real applications, you normally use a **Deployment**, not a standalone Pod.

Example:

apiVersion: apps/v1

kind: Deployment

metadata:

name: secret-demo-app

namespace: secret-demo

spec:

replicas: 2

selector:

matchLabels:

app: secret-demo

template:

metadata:

labels:

app: secret-demo

spec:

containers:

- name: app

image: nginx:latest

env:

- name: DB_USERNAME

valueFrom:

secretKeyRef:

name: app-secret

key: username

```
- name: DB_PASSWORD
```

```
valueFrom:
```

```
secretKeyRef:
```

```
name: app-secret
```

```
key: password
```

Apply:

```
kubectl apply -f secret-deployment.yaml
```

Check:

```
kubectl get pods -n secret-demo
```

You should get two Pods:

```
secret-demo-app-xxxxx 1/1 Running
```

```
secret-demo-app-yyyyy 1/1 Running
```

Both Pods receive the Secret.

⚠ Important: Don't Put Secrets Directly in Git

Avoid committing this:

```
data:
```

```
password: TXlQQHNzMTIz
```

Even though it looks encoded, it can easily be decoded.

For production environments, commonly consider:

AWS Secrets Manager

```
|
```

```
v
```

External Secrets Operator

```
|
```

```
v
```

Kubernetes Secret

|

v

Application Pod

For EKS, this is a much more production-oriented architecture than manually committing Kubernetes Secret YAML files into Git.

Step 17: Check Secret permissions

Run:

```
kubectrl auth can-i get secrets -n secret-demo
```

Output:

yes

This is important because Kubernetes **RBAC** controls who can read Secret objects.

Step 18: Troubleshooting

Problem 1: Pod is not starting

```
kubectrl describe pod secret-env-pod -n secret-demo
```

Look at:

Events:

Problem 2: Secret doesn't exist

Check:

```
kubectrl get secret -n secret-demo
```

If you see:

Error from server (NotFound)

make sure the Secret name matches:

secretKeyRef:

name: app-secret

Problem 3: Key doesn't exist

Suppose your Secret contains:

username

password

But your YAML says:

key: user

You'll get an error because:

user != username

Check:

kubectl describe secret app-secret -n secret-demo

Problem 4: Secret is in another namespace

This will NOT work:

Secret → namespace A

Pod → namespace B

A Pod can only reference a Secret **in the same namespace**.

For example:

secret-demo

|

+-- app-secret

|

+-- secret-env-pod

Step 19: Cleanup

When your lab is complete:

```
kubectl delete namespace secret-demo
```

Verify:

```
kubectl get namespace secret-demo
```

Expected:

Error from server (NotFound): namespaces "secret-demo" not found

□ Interview Questions

1. What is a Kubernetes Secret?

A Kubernetes Secret is an API object used to store sensitive configuration such as passwords, tokens, API keys and certificates.

2. Is Base64 encryption?

No. Base64 is encoding, not encryption.

3. How can you inject a Secret into a Pod?

Two common methods:

1. Environment variables
2. Volume mounts

4. Can a Pod in namespace A access a Secret in namespace B?

No. Secrets are namespace-scoped.

5. Secret vs ConfigMap?

ConfigMap

↓

Non-sensitive configuration

Secret



Sensitive configuration

Examples:

ConfigMap:

APP_MODE=production

LOG_LEVEL=INFO

Secret:

DB_PASSWORD=xxxx

API_TOKEN=xxxx

6. Does Kubernetes Secret automatically mean the data is securely encrypted everywhere?

No. Base64 encoding is not encryption. Cluster administrators should configure appropriate encryption at rest, RBAC, and secure secret-management practices.

Your Hands-on Lab Flow

Run these in order:

```
kubectl create namespace secret-demo
```

```
kubectl create secret generic app-secret \
```

```
-n secret-demo \
```

```
--from-literal=username=admin \
```

```
--from-literal=password='MyP@ss123'
```

```
kubectl get secret -n secret-demo
```

Create the Pod:

```
kubectl apply -f secret-env-pod.yaml
```

Check:

```
kubectl get pods -n secret-demo
```

Then:

```
kubectl exec -n secret-demo secret-env-pod -- env | grep DB_
```

You should see:

```
DB_USERNAME=admin
```

```
DB_PASSWORD=MyP@ss123
```

Then test the volume method:

```
kubectl apply -f secret-volume-pod.yaml
```

```
kubectl exec -n secret-demo secret-volume-pod -- ls /etc/app-secret
```

You should see:

```
password
```

```
username
```

That completes Lab #6: Kubernetes Secret → inject into Pod.

7. Check different add-ons available on EKS.

Step 1: Check your EKS cluster

Since you're using us-east-1, first verify the cluster:

```
aws eks list-clusters --region us-east-1
```

Then check your Kubernetes version:

```
kubectl version
```

Or:

```
aws eks describe-cluster \
  --name dev-eks-cluster \
  --region us-east-1 \
  --query "cluster.version" \
  --output text
```

You should get something like:

1.34

Step 2: Check add-ons currently installed

This is the **first command I recommend for your lab**:

```
aws eks list-addons \
  --cluster-name dev-eks-cluster \
  --region us-east-1
```

AWS's list-addons API shows the add-ons currently installed on the specified EKS cluster.

You might see:

```
{
  "addons": [
    "coredns",
    "kube-proxy",
    "vpc-cni",
    "aws-ebs-csi-driver"
  ]
}
```

Your actual output may be different.

Step 3: Check add-ons from Kubernetes

You can also check the corresponding Pods:

```
kubectl get pods -n kube-system
```

For example:

NAME	READY	STATUS
------	-------	--------

aws-node-xxxxx	2/2	Running
coredns-xxxxx	1/1	Running
kube-proxy-xxxxx	1/1	Running
ebs-csi-controller-xxxxx	6/6	Running
ebs-csi-node-xxxxx	3/3	Running
metrics-server-xxxxx	1/1	Running

Step 4: Important EKS Add-ons

Here are the major ones you should know for interviews and practical work.

Add-on	Purpose
VPC CN	Pod networking
CoreDNS	Kubernetes DNS/service discovery
kube-proxy	Service networking
EBS CSI Driver	EBS volumes
EFS CSI Driver	Shared EFS storage
FSx CSI Driver	FSx storage
S3 CSI Driver	Mount/access S3 from Pods
CSI Snapshot Controller	Volume snapshots
EKS Pod Identity Agent	Gives Pods AWS IAM permissions
CloudWatch Observability	Logs/metrics/observability
AWS Distro for OpenTelemetry	Metrics/traces/telemetry
GuardDuty Agent	Runtime security monitoring
Node Monitoring Agent	Detect node health issues

AWS's current EKS add-on catalog includes these and additional AWS add-ons.

1. VPC CNI

Add-on name:

vpc-cni

This is one of the most important EKS components.

It provides **native AWS VPC networking for Pods**.

Pod

|

| private IP

v

AWS VPC

|

v

EC2 / AWS resources

Check it:

```
kubectl get pods -n kube-system -l k8s-app=aws-node
```

You should see:

```
aws-node-xxxxx 2/2 Running
```

AWS installs VPC CNI by default for traditional EC2-based EKS clusters.

2. CoreDNS

Add-on:

coredns

CoreDNS provides **DNS/service discovery** inside Kubernetes.

For example:

```
my-service.default.svc.cluster.local
```

resolves to the Kubernetes Service IP.

Check:

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
```

You may see:

```
coredns-xxxxx 1/1 Running
```

```
coredns-yyyyy 1/1 Running
```

AWS deploys CoreDNS when an EKS cluster has nodes.

3. kube-proxy

Add-on:

kube-proxy

It maintains networking rules on EC2 nodes and enables communication with Kubernetes Services and Pods.

Check:

```
kubectl get pods -n kube-system -l k8s-app=kube-proxy
```

Example:

```
kube-proxy-xxxxx 1/1 Running
```

```
kube-proxy-yyyyy 1/1 Running
```

Usually there is one kube-proxy Pod per EC2 node.

4. Amazon EBS CSI Driver

Add-on:

aws-ebs-csi-driver

This allows Kubernetes to use **Amazon EBS volumes**.

Architecture:

Pod

|

PVC

|

PV

|

EBS CSI Driver

|

Amazon EBS

Check:

```
kubectl get pods -n kube-system | grep ebs
```

You may see:

```
ebs-csi-controller-xxxxx
```

```
ebs-csi-node-xxxxx
```

This is especially important for Stateful applications such as:

MySQL

PostgreSQL

Jenkins

MongoDB

AWS provides the EBS CSI driver as an EKS add-on.

5. Amazon EFS CSI Driver

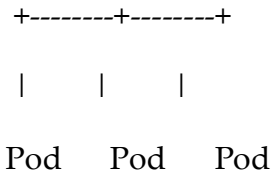
Add-on:

```
aws-efs-csi-driver
```

EFS is useful when multiple Pods need shared storage.

Amazon EFS

|



For example:

Pod 1 → Read/Write

Pod 2 → Read/Write

Pod 3 → Read/Write

Check:

```
kubectrl get pods -n kube-system | grep efs
```

6. Metrics Server

This one is especially important for your **HPA lab**.

Check:

```
kubectrl get pods -n kube-system | grep metrics
```

You already have a Metrics Server Pod in your current EKS work, so you can verify it with:

```
kubectrl top nodes
```

and:

```
kubectrl top pods -A
```

Example:

NAME	CPU(cores)	CPU%
ip-10-0-8-xxx	120m	6%
ip-10-0-9-xxx	150m	7%

Metrics Server provides resource metrics used by Kubernetes autoscaling mechanisms such as HPA. AWS currently lists Metrics Server under **community add-ons**, rather than AWS-built add-ons.

7. EKS Pod Identity Agent

Add-on:

eks-pod-identity-agent

This allows Kubernetes workloads to obtain AWS permissions without putting AWS access keys inside Pods.

Architecture:

Pod

|

| AWS API request

v

EKS Pod Identity

|

v

IAM Role

|

v

AWS Service

For example:

Pod → S3

Pod → DynamoDB

Pod → Secrets Manager

Pod → SQS

This is an important production security concept.

8. AWS CloudWatch Observability

This add-on is useful for monitoring and logging.

Architecture:

EKS

|

+-- Logs

+-- Metrics

+-- Container information

|

v

CloudWatch

Useful for production troubleshooting.

9. AWS Distro for OpenTelemetry

Add-on:

adot

Used for observability such as:

Metrics

Traces

Telemetry

It can be used as part of an OpenTelemetry-based monitoring architecture.

10. GuardDuty Agent

Add-on:

aws-guardduty-agent

Used for runtime security monitoring of workloads/nodes.

Architecture:

EKS Nodes

```
|
v
GuardDuty Agent
|
v
Amazon GuardDuty
|
v
Security Findings
```

Step 5: See the complete list available for YOUR Kubernetes version

This is very important.

Don't assume that every add-on is available for every Kubernetes version.

Run:

```
eksctl utils describe-addon-versions \
--kubernetes-version 1.34
```

If your cluster is a different version, replace 1.34.

For example:

```
eksctl utils describe-addon-versions \
--kubernetes-version 1.34 \
| grep AddonName
```

AWS documents this method for checking add-ons compatible with a specific Kubernetes version.

You might see:

```
"AddonName": "aws-ebs-csi-driver"
```

```
"AddonName": "coredns"
```

```
"AddonName": "kube-proxy"
```

```
"AddonName": "vpc-cni"
```

```
"AddonName": "adot"
```

```
...
```

Step 6: Check one add-on's available versions

For example, VPC CNI:

```
eksctl utils describe-addon-versions \
```

```
--kubernetes-version 1.34 \
```

```
--name vpc-cni
```

For CoreDNS:

```
eksctl utils describe-addon-versions \
```

```
--kubernetes-version 1.34 \
```

```
--name coredns
```

For EBS CSI:

```
eksctl utils describe-addon-versions \
```

```
--kubernetes-version 1.34 \
```

```
--name aws-ebs-csi-driver
```

This is better than blindly selecting an add-on version because EKS add-on versions are tied to Kubernetes compatibility.

Step 7: Check add-on details

For example:

```
aws eks describe-addon \
```

```
--cluster-name dev-eks-cluster \
```

```
--addon-name vpc-cni \
```

```
--region us-east-1
```

You can also check:

```
aws eks describe-addon \  
  --cluster-name dev-eks-cluster \  
  --addon-name coredns \  
  --region us-east-1
```

Step 8: Check add-ons in AWS Console

Go to:

AWS Console → **EKS** → **Clusters** → **dev-eks-cluster**

Then:

dev-eks-cluster

|

+-- Overview

+-- Compute

+-- Add-ons ← Click here

+-- Networking

+-- Storage

+-- Observability

Under **Add-ons**, you can see:

Name	Status
------	--------

Amazon VPC CNI	Active
----------------	--------

CoreDNS	Active
---------	--------

kube-proxy	Active
------------	--------

Amazon EBS CSI Driver	Active
-----------------------	--------

Metrics Server Active

...

AWS recommends using managed EKS add-ons where possible because AWS can manage their installation, configuration, and updates.

□ Your Hands-On Lab

Run these commands one by one:

1. List installed add-ons

```
aws eks list-addons \
  --cluster-name dev-eks-cluster \
  --region us-east-1
```

2. Check Kubernetes add-on Pods

```
kubectl get pods -n kube-system
```

3. Check VPC CNI

```
kubectl get pods -n kube-system -l k8s-app=aws-node
```

4. Check CoreDNS

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
```

5. Check kube-proxy

```
kubectl get pods -n kube-system -l k8s-app=kube-proxy
```

6. Check storage drivers

```
kubectl get pods -n kube-system | grep -E 'ebs|efs|fsx'
```

7. Check Metrics Server

```
kubectl get pods -n kube-system | grep metrics
```

8. Check metrics

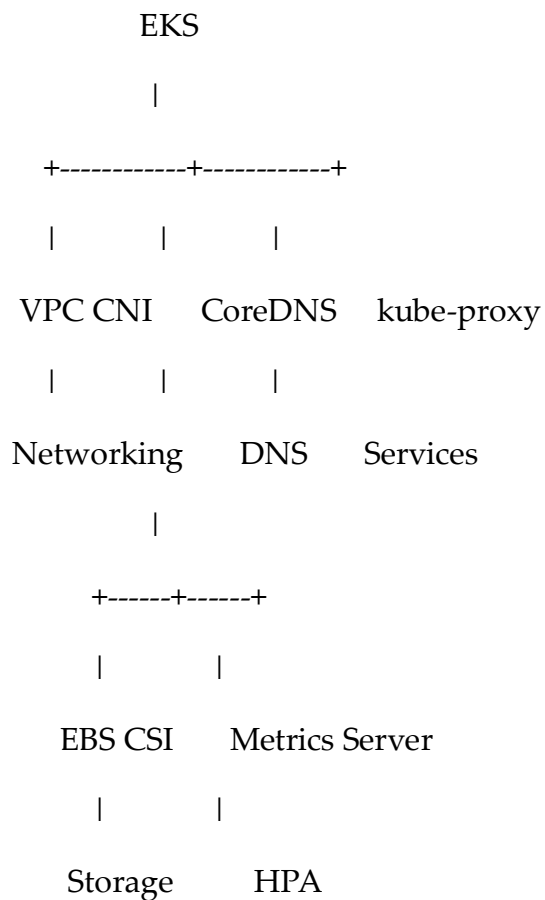
```
kubectl top nodes
```

9. List add-ons compatible with your Kubernetes version

```
eksctl utils describe-addon-versions \
--kubernetes-version 1.34 \
| grep AddonName
```

☆ Interview Summary

Remember these **5 core EKS components**:



One-line interview answer:

EKS add-ons are Kubernetes operational components that provide networking, DNS, service networking, storage, monitoring, security, and observability capabilities. Core EKS add-ons include VPC CNI, CoreDNS, kube-proxy, and commonly EBS CSI; additional AWS and community add-ons can be installed according to workload requirements.

8.Upgrade eks cluster

Step 1 — Check your current cluster

Your cluster from the previous labs is:

dev-eks-cluster

Run:

```
aws eks describe-cluster \
  --name dev-eks-cluster \
  --region us-east-1 \
  --query "cluster.version" \
  --output text
```

Example:

1.34

Also check nodes:

```
kubectl get nodes -o wide
```

Example:

NAME	STATUS	VERSION
ip-10-0-8-xxx.ec2.internal	Ready	v1.34.x
ip-10-0-9-xxx.ec2.internal	Ready	v1.34.x

Step 2 — Check EKS add-ons

Before upgrading, check what you have installed:

```
aws eks list-addons \
  --cluster-name dev-eks-cluster \
  --region us-east-1
```

For example:

```
{
  "addons": [
```

```
"vpc-cni",  
"coredns",  
"kube-proxy",  
"aws-ebs-csi-driver"  
]  
}
```

Remember:

EKS does NOT automatically upgrade your EKS add-ons when you upgrade the Kubernetes control plane. You must update them separately.

Step 3 — Check upgrade readiness

AWS recommends checking **EKS Upgrade Insights** before upgrading. It helps identify workloads using APIs or configurations that could cause upgrade problems.

From the AWS Console:

AWS Console



EKS



Clusters



dev-eks-cluster



Upgrade insights

Review any warnings before continuing.

Step 4 — Check workloads

Run:

```
kubectl get pods -A
```

Then:

```
kubectl get deployments -A
```

And:

```
kubectl get statefulsets -A
```

Also:

```
kubectl get pvc -A
```

You want to make sure important applications are healthy.

Step 5 — Check node versions

Run:

```
kubectl get nodes
```

You might see:

NAME	STATUS	ROLES	VERSION
ip-10-0-8-xxx.ec2.internal	Ready	<none>	v1.34.x
ip-10-0-9-xxx.ec2.internal	Ready	<none>	v1.34.x

Ideally, upgrade your managed node groups so they match the control plane. AWS specifically recommends keeping the data plane at the same minor version as the control plane.

Step 6 — Check your EKS node groups

Run:

```
aws eks list-nodegroups \
  --cluster-name dev-eks-cluster \
  --region us-east-1
```

Example:

```
{
```

```
"nodegroups": [  
  "ng-1"  
]  
}
```

Then:

```
aws eks describe-nodegroup \  
  --cluster-name dev-eks-cluster \  
  --nodegroup-name ng-1 \  
  --region us-east-1 \  
  --query "nodegroup.version" \  
  --output text
```

Example:

1.34

Step 7 — Check available target versions

Don't assume the target version.

Run:

```
aws eks describe-addon-versions \  
  --kubernetes-version 1.35 \  
  --region us-east-1 \  
  --query "addons[].addonName" \  
  --output text
```

And check EKS supported versions through the AWS documentation/console before selecting your target.

For this lab, if your cluster is currently:

1.34

we'll use:

1.35

Step 8 — Upgrade the EKS Control Plane

You have two options.

Option A — AWS Console

Go to:

AWS Console

↓

EKS

↓

Clusters

↓

dev-eks-cluster

Choose:

Upgrade now

Select:

1.35

Then:

Upgrade

AWS will upgrade the EKS control plane. The control plane upgrade normally takes several minutes.

Option B — AWS CLI

I recommend this for your hands-on lab because you will understand what is happening.

Run:

```
aws eks update-cluster-version \  
  --name dev-eks-cluster \  
  --kubernetes-version 1.35 \  
  --region us-east-1
```

You'll receive something similar to:

```
{  
  "update": {  
    "id": "xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxxxx",  
    "status": "InProgress",  
    "type": "VersionUpdate"  
  }  
}
```

Step 9 — Monitor the upgrade

Get the update ID from the previous command and run:

```
aws eks describe-update \  
  --name dev-eks-cluster \  
  --region us-east-1 \  
  --update-id UPDATE_ID
```

For example:

```
aws eks describe-update \  
  --name dev-eks-cluster \  
  --region us-east-1 \  
  --update-id abc123
```

Initially:

```
status: InProgress
```

Eventually:

status: Successful

AWS documents this as the standard way to monitor an EKS control-plane version update.

Step 10 — Verify Control Plane

Run:

```
aws eks describe-cluster \
  --name dev-eks-cluster \
  --region us-east-1 \
  --query "cluster.version" \
  --output text
```

Expected:

1.35

🔔 Your EKS control plane is now 1.35.

But we are **not finished yet**.

Step 11 — Important: Nodes are still old

Run:

```
kubectl get nodes
```

You might see:

NAME	STATUS	VERSION
ip-10-0-8-xxx.ec2.internal	Ready	v1.34.x
ip-10-0-9-xxx.ec2.internal	Ready	v1.34.x

This is normal.

The EKS control plane upgrade **does not automatically upgrade a normal managed node group**.

So now we need to upgrade the worker nodes.

Step 12 — Upgrade Managed Node Group

First get the node group name:

```
aws eks list-nodegroups \  
  --cluster-name dev-eks-cluster \  
  --region us-east-1
```

Suppose:

ng-1

You can upgrade it using eksctl:

```
eksctl upgrade nodegroup \  
  --name=ng-1 \  
  --cluster=dev-eks-cluster \  
  --region=us-east-1 \  
  --kubernetes-version=1.35
```

Or using AWS CLI:

```
aws eks update-nodegroup-version \  
  --cluster-name dev-eks-cluster \  
  --nodegroup-name ng-1 \  
  --kubernetes-version 1.35 \  
  --region us-east-1
```

Amazon EKS performs a managed rolling update, draining nodes and replacing/updating them according to the node group's update configuration.

Step 13 — Watch the nodes

Run:

```
kubectl get nodes -w
```

During the update you may see nodes become:

Ready

then get replaced.

Eventually:

NAME	STATUS	VERSION
ip-10-0-8-xxx.ec2.internal	Ready	v1.35.x
ip-10-0-9-xxx.ec2.internal	Ready	v1.35.x

Press:

Ctrl+C

to stop watching.

Step 14 — Verify node group version

Run:

```
aws eks describe-nodegroup \  
  --cluster-name dev-eks-cluster \  
  --nodegroup-name ng-1 \  
  --region us-east-1 \  
  --query "nodegroup.version" \  
  --output text
```

Expected:

1.35

Step 15 — Upgrade EKS Add-ons

Now check:

```
aws eks list-addons \  

```

```
--cluster-name dev-eks-cluster \
```

```
--region us-east-1
```

Typical add-ons:

vpc-cni

coredns

kube-proxy

aws-ebs-csi-driver

Remember:

Control Plane Upgrade

↓

Does NOT automatically upgrade

↓

EKS Add-ons

AWS explicitly requires you to initiate add-on updates.

Step 16 — Find compatible add-on versions

For example:

```
aws eks describe-addon-versions \
```

```
--cluster-name dev-eks-cluster \
```

```
--kubernetes-version 1.35 \
```

```
--addon-name vpc-cni \
```

```
--region us-east-1
```

You can do the same for CoreDNS:

```
aws eks describe-addon-versions \
```

```
--cluster-name dev-eks-cluster \
```

```
--kubernetes-version 1.35 \
```

```
--addon-name coredns \
```

```
--region us-east-1
```

And kube-proxy:

```
aws eks describe-addon-versions \
```

```
--cluster-name dev-eks-cluster \
```

```
--kubernetes-version 1.35 \
```

```
--addon-name kube-proxy \
```

```
--region us-east-1
```

Step 17 — Update an add-on

First determine the recommended compatible version.

Then:

```
aws eks update-addon \
```

```
--cluster-name dev-eks-cluster \
```

```
--addon-name vpc-cni \
```

```
--addon-version ADDON_VERSION \
```

```
--region us-east-1 \
```

```
--resolve-conflicts PRESERVE
```

For CoreDNS:

```
aws eks update-addon \
```

```
--cluster-name dev-eks-cluster \
```

```
--addon-name coredns \
```

```
--addon-version ADDON_VERSION \
```

```
--region us-east-1 \
```

```
--resolve-conflicts PRESERVE
```

For kube-proxy:

```
aws eks update-addon \  
  --cluster-name dev-eks-cluster \  
  --addon-name kube-proxy \  
  --addon-version ADDON_VERSION \  
  --region us-east-1 \  
  --resolve-conflicts PRESERVE
```

Don't copy a random add-on version from an old tutorial; select one compatible with your target Kubernetes version.

Step 18 — Check add-on status

Run:

```
aws eks describe-addon \  
  --cluster-name dev-eks-cluster \  
  --addon-name vpc-cni \  
  --region us-east-1 \  
  --query "addon.status"
```

Expected:

"ACTIVE"

Do the same for:

coredns

kube-proxy

aws-ebs-csi-driver

Step 19 — Update kubectl

Check:

```
kubectl version --client
```

AWS recommends using a kubectl version within one minor version of the EKS control plane.

For a 1.35 cluster, use a compatible kubectl release.

Step 20 — Final health check

Now run everything together:

```
kubectl get nodes
```

Expected:

NAME	STATUS	VERSION
ip-10-0-8-xxx.ec2.internal	Ready	v1.35.x
ip-10-0-9-xxx.ec2.internal	Ready	v1.35.x

Then:

```
kubectl get pods -A
```

All important Pods should be:

Running

or expected states such as:

Completed

Check system Pods:

```
kubectl get pods -n kube-system
```

Check metrics:

```
kubectl top nodes
```

Check add-ons:

```
aws eks list-addons \
  --cluster-name dev-eks-cluster \
  --region us-east-1
```

Complete EKS Upgrade Flow

Remember this sequence for interviews:

EKS UPGRADE

|

v

1. Check current version

|

v

2. Check Upgrade Insights

|

v

3. Check workloads

|

v

4. Backup / prepare

|

v

5. Upgrade CONTROL PLANE

|

v

6. Upgrade NODE GROUPS

|

v

7. Upgrade EKS ADD-ONS

|

v

8. Upgrade kubectl

|

v

9. Verify workloads

|

v

UPGRADE COMPLETE

